

SDMIT-UJIRE

Web-Technology and Applications (Module1)

Module -1

**By Harsha Nagaraj Honnu ,Assistant Professor, Department Of
Information Science and Engineering,**

Introduction to HTML, What is HTML and Where did it come from?, HTML Syntax, Semantic Markup, Structure of HTML Documents, Quick Tour of HTML Elements, HTML5 Semantic Structure Elements, Introduction to CSS, What is CSS, CSS Syntax, Location of Styles, Selectors, The Cascade: How Styles Interact, The Box Model, CSS Text Styling.

HTML

Brief history of HTML:

HTML, or Hypertext Markup Language, has come a long way since its inception. It all began with ARPANET in the late 1960s, which laid the foundation for the internet as we know it today. However, it wasn't until 1991 that Tim Berners-Lee introduced the first public specification of HTML, which revolutionized the way we interact with the web.

Fast forward to 1997, when the World-Wide Web Consortium (W3C) codified HTML, making it the standard language for web development. This move paved the way for the creation of dynamic and interactive websites that we see today.

In conclusion, HTML has undergone significant changes over the years, and its evolution has played a crucial role in shaping the internet as we know it.

HTML Syntax

HTML, or Hypertext Markup Language, is a language used for creating web pages. It is a markup language, which means that it is a way of annotating a document to make the annotations distinct from the text being annotated. This is done by using tags, which are enclosed in angle brackets, to define the structure and content of the page.

The term "markup language" comes from the days of print, when editors would write instructions on manuscript pages that might be revision instructions to the author or copy editor. In the same way, HTML tags provide instructions to web browsers on how to display the content of a web page.

HTML syntax is relatively simple, but it is important to use it correctly in order to create well-formed web pages that are easy to read and navigate. This involves using the correct tags in the correct order, and ensuring that all tags are properly closed.

By mastering HTML syntax, you can create professional-looking web pages that are easy to use and navigate. Whether you are creating a personal blog or a corporate website, HTML is an essential tool for anyone who wants to create content for the web.

Web-Technology and Applications (Module1)

Markup is a fundamental concept in web development that allows developers to add information about the content of a webpage. This information is implemented through the use of tags, also known as elements, in HTML.

To put it simply, markup is a way to provide context and structure to the content on a webpage. It helps search engines understand the content and aids in accessibility for users with disabilities. In the previous slide, you may have noticed the red text, circles, arrows, and yellow sticky notes. These are all examples of markup that can be used to enhance the presentation and organization of content on a webpage.

HTML, the language used to create WebPages, utilizes textual tags to achieve the same effect. By using these tags, developers can create a well-structured and easily navigable website that provides a seamless user experience.

What is W3C?

The W3C, or World Wide Web Consortium, is the primary standards organization for the internet. Its mission is to promote compatibility and interoperability among web technologies. To achieve this goal, the W3C produces recommendations, also known as specifications that define how web technologies should work.

One of the most significant specifications developed by the W3C is XHTML 1.0. This version of HTML uses stricter XML syntax rules, which makes it more reliable and easier to maintain. The W3C began focusing on XHTML 1.0 in 1998, recognizing its potential to improve the web's overall quality and consistency.

Overall, the W3C plays a crucial role in shaping the internet as we know it today. Its work ensures that web technologies are accessible, reliable, and compatible across different platforms and devices. As such, it is an essential organization for anyone involved in web development or online business.

XHTML

Extensible Hypertext Markup Language, was developed with the aim of making web page rendering more predictable. This was achieved by imposing strict rules on web authors, which required them to create web pages without any syntax errors.

Web-Technology and Applications (Module1)

The syntax rules for XHTML are based on XML and are relatively easy to follow. The primary rules include using lowercase tag names, enclosing attributes within quotes, and ensuring that all elements have a closing element or are self-closing.

By adhering to these rules, web authors can ensure that their web pages are compatible with a wide range of devices and browsers, making them more accessible to users. So, if you want to create web pages that are both professional and user-friendly, you too can adopt the strict standards of XHTML.

XHTML, or Extensible Hypertext Markup Language, was developed to provide a more structured and standardized approach to web page design. To assist web authors in creating compliant web pages, two versions of XHTML were introduced: **XHTML 1.0 Strict** and **XHTML 1.0 Transitional**.

The strict version of XHTML was designed to be rendered by web browsers that adhere to the strict syntax rules and tag support outlined in the W3C XHTML 1.0 Strict specification. This version is ideal for web developers who want to ensure that their web pages are fully compliant with the latest web standards.

On the other hand, the transitional recommendation is a more lenient version of XHTML that allows for some flexibility in syntax and tag usage. This version was intended to serve as a temporary transition to the eventual global adoption of XHTML Strict.

In summary, XHTML provides web authors with a standardized approach to web page design, and the two versions of XHTML offer varying levels of compliance and flexibility to suit different web development needs.

The Standards Movement: Embracing Best Practices

Adhering to standards is a crucial aspect of web development. In the early 2000s, the professional web development community placed a significant emphasis on following standards, particularly the W3C specification for XHTML. This approach aimed to ensure that websites were built using best practices, resulting in improved functionality, accessibility, and user experience.

By embracing standards, developers could create websites that were compatible with a wide range of devices and browsers, making them accessible to a broader audience. Additionally,

adhering to standards helped to future-proof websites, ensuring that they would remain functional and relevant for years to come.

Today, the importance of standards remains as relevant as ever. As technology continues to evolve, it is essential to stay up-to-date with the latest best practices to ensure that websites remain accessible, functional, and user-friendly. By embracing the Standards Movement, web developers can create websites that meet the needs of their clients and users, while also staying ahead of the curve in an ever-changing digital landscape.

Validators

Validators are an essential component of the web development community's standards movement in the 2000s. HTML Validators were widely used to verify that a web page's markup adhered to the rules for XHTML transitional or strict. This process ensured that web pages followed a standard and were compatible with various browsers and devices.

To ensure that your web pages follow a standard, it is crucial to use HTML Validators. These tools help identify errors in your markup and provide suggestions for improvement. By using Validators, you can ensure that your web pages are accessible, user-friendly, and compatible with different devices and browsers.

In summary, Validators play a vital role in the web development community by ensuring that web pages adhere to standards and are accessible to all users. By using Validators, you can improve the quality of your web pages and provide a better user experience.

The screenshot shows the W3C Markup Validation Service homepage. At the top, there's a dark blue header with the W3C logo and the text 'Markup Validation Service'. Below this, there are three tabs: 'Validate by URI' (selected), 'Validate by File Upload', and 'Validate by Direct Input'. Under the 'Validate by URI' tab, there's a text input field for the 'Address' and a 'Check' button. Below the input field, there's a link for 'More Options'. A paragraph of text explains the validator's capabilities and provides links for more information. At the bottom, there's a green banner with a 'Donate' button and a footer with navigation links and copyright information.

W3C[®] Markup Validation Service
Check the markup (HTML, XHTML, ...) of Web documents

Validate by URI Validate by File Upload Validate by Direct Input

Validate by URI

Validate a document online:

Address:

• More Options

Check

This validator checks the [markup validity](#) of Web documents in HTML, XHTML, SMIL, MathML, etc. If you wish to validate specific content such as [RSS/Atom feeds](#) or [CSS stylesheets](#), [MobileOK content](#), or to [find broken links](#), there are other [validators and tools](#) available. As an alternative you can also try our [non-DTD-based validator](#).

The W3C validators rely on community support for hosting and development. [Donate](#) and help us build better tools for a better web.

3622 [Flats](#)

Home About... News Docs Help & FAQ Feedback Contribute

W3C[®] open source

This service runs the W3C Markup Validator [v1.3](#)
COPYRIGHT © 1994-2012 W3C (MIT, ERIC, AND OI). ALL RIGHTS RESERVED. W3C LOGO, TRADEMARK, DOCUMENT USE AND SOFTWARE LICENSING RULES APPLY. YOUR INTERACTIONS WITH THIS SITE ARE IN ACCORDANCE WITH OUR PUBLIC AND MEMBER PRIVACY STATEMENTS

W3C[®]

Would you like an example? Keep in mind that an internet connection is required for this task.

Here are the steps to follow:

1. Open a web browser and navigate to the W3C validator website.
2. Choose a few websites to test.
3. Enter the website's URL into the designated bar.
4. The validator will then check if the website's home page is valid against various standards or auto-detect any issues.

By following these simple steps, you can easily test the validity of any website. This is a crucial step in ensuring that your website is professional, user-friendly, and accessible to all users.

To ensure the quality and compliance of websites, it is essential to use the W3C validator. To get started, simply open a web browser and navigate to the W3C validator. From there, select a few websites to test and enter their URLs into the designated bar. With just a few clicks, you can easily check if the home page is valid against various standards or use the auto-detect feature for added convenience. By utilizing this powerful tool, you can ensure that your website meets the highest standards of quality and accessibility.

XHTML2.0 and WHATWG

In the mid-2000s, the web development world was abuzz with the proposed changes to HTML brought about by XHTML 2.0. This revolutionary proposal aimed to make significant changes to the existing HTML standards, dropping backwards compatibility with HTML and XHTML 1.0. The proposed changes also meant that browsers would become less forgiving of invalid markup, which would have a significant impact on web development.

Around the same time, a group of developers at Opera and Mozilla formed the WHATWG (Web Hypertext Application Technology Working Group) within the W3C. This group was not convinced that the W3C's embrace of XML and its abandonment of backwards-compatibility was the best way forward for the web. The WHATWG aimed to create a more practical and flexible approach to web development that would be more accessible to developers and users alike.

In summary, the proposed changes to HTML brought about by XHTML 2.0 and the formation of the WHATWG were significant developments in the world of web development. These changes aimed to create a more practical and flexible approach to web development that would benefit developers and users alike.

Html

In 2009, the World Wide Web Consortium (W3C) ceased development on XHTML 2.0 and opted to embrace the work of the Web Hypertext Application Technology Working Group (WHATWG), which they named HTML5. This decision was made with three primary objectives in mind:

1. To establish clear guidelines for how web browsers should handle invalid markup, ensuring consistency and reliability across all platforms.
2. To provide an open, non-proprietary programming framework through the use of Javascript, enabling developers to create dynamic and engaging web applications.
3. To maintain backward compatibility with existing web technologies, allowing for a seamless transition to HTML5 without disrupting the user experience.

Overall, HTML5 represents a significant step forward in the evolution of web development, offering a powerful and flexible platform for creating innovative and engaging online experiences.

Web-Technology and Applications (Module1)

Although certain aspects of HTML5 are still being finalized, all major browser manufacturers have already partially embraced this new technology. However, it's important to note that not all browsers and versions support every feature of HTML5. This is actually intentional, as HTML in HTML5 is now considered a living language that evolves and develops over time. As a result, each browser will support a gradually increasing subset of HTML5 capabilities.

In other words, HTML5 is not a static language that remains the same over time. Instead, it's a dynamic and ever-changing language that adapts to the needs of modern web development. This means that while some browsers may not support certain features of HTML5 right now, they will likely do so in the future as the language continues to evolve.

Overall, HTML5 represents a major step forward in web development, offering a range of new features and capabilities that were not possible with previous versions of HTML. As more and more browsers embrace this new technology, we can expect to see even more exciting developments in the world of web design and development.

HTML SYNTAX

Elements and attributes

HTML documents consist of textual content and HTML elements. These elements can contain text, other elements, or be empty, and are identified in the document by tags. Additionally, HTML elements can contain attributes, which provide more information about the element through a name=value pair.

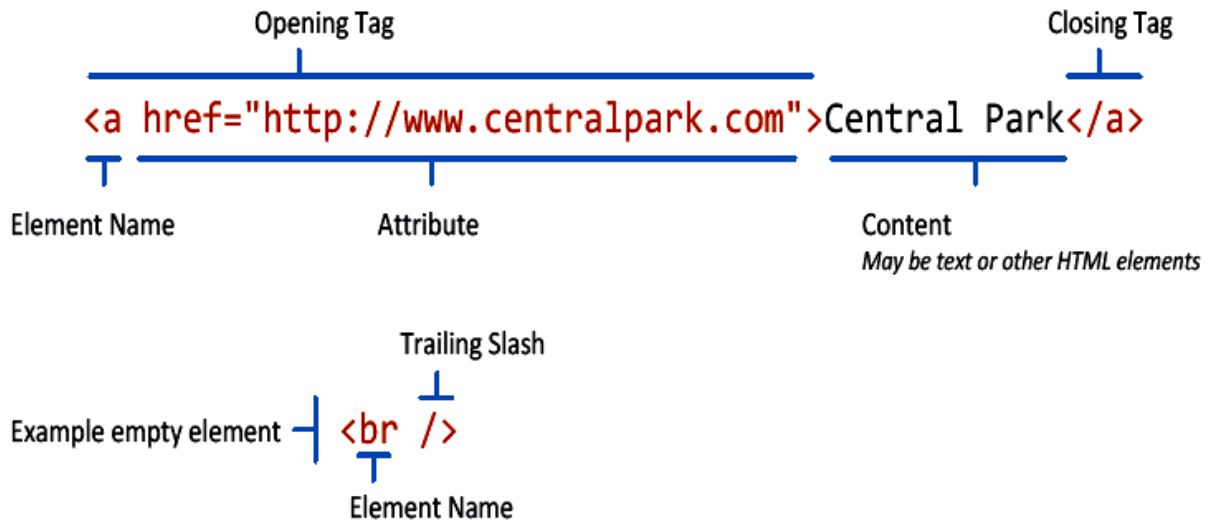
It's important to note that in XHTML, attribute values had to be enclosed in quotes. However, in HTML5, the quotes are now optional. This change allows for more flexibility in coding and streamlines the process for developers.

Understanding the basics of HTML elements and attributes is crucial for creating effective and visually appealing web pages. By utilizing these tools, developers can create dynamic and engaging content that captures the attention of their audience.

HTML, or Hypertext Markup Language, is a powerful tool that allows you to create visually appealing and interactive web pages. With HTML, you can insert images using the `<img>` tag, create links with the `<a>` tag, and organize content with lists using the `<ul>`, `<ol>`, and `<li>` tags. Additionally, you can create headings with `<H1>`, `<H2>`, ..., `<H6>` tags, and define metadata with the `<meta>` tag. The possibilities are endless with HTML, and you can do much more than

Web-Technology and Applications (Module1)

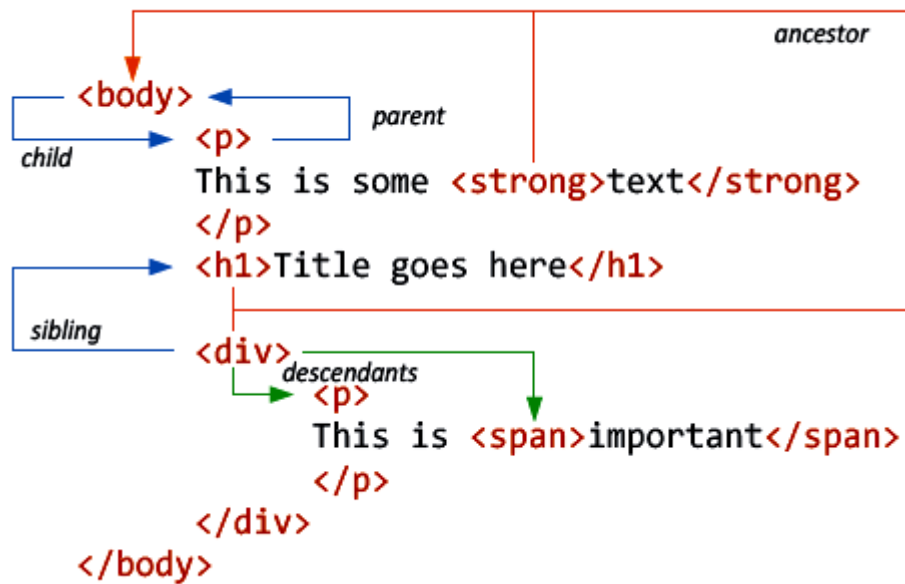
just these basic functions. By mastering HTML, you can create dynamic and engaging web pages that will captivate your audience.



Nesting HTML Elements:

Frequently, an HTML element will enclose other HTML elements. In this scenario, the enclosing element is referred to as the parent of the enclosed, or child, element. Any elements that are enclosed within the child are considered to be descendants of the parent element. Similarly, any given child element may have a range of ancestors.

HTML elements are often nested to create a structured and organized layout for a webpage. This technique allows for the creation of complex designs and functionalities. Understanding the concept of parent-child relationships and descendant elements is crucial for web developers to create effective and efficient web pages. By utilizing nesting HTML elements, developers can create visually appealing and user-friendly websites.



Nesting of HTML elements

To create a well-organized hierarchy of elements, it is crucial to ensure that each HTML nested element is properly nested. This means that the ending tag of a child element must come before the ending tag of its parent element. Your browser relies on this structure to accurately display the content of your webpage. So, make sure to double-check your code and ensure that your elements are nested correctly to avoid any display issues.

Semantic markup

Over the past decade, there has been a growing consensus that HTML documents should focus solely on the structure of the content. This means that information about how the content should look when displayed in the browser is best left to Cascading Style Sheets (CSS). As a result, novice HTML authors are often advised to create semantic HTML documents. In other words, an HTML document should not describe how to visually present content, but rather describe its content's structural semantics or meaning.

Semantic markup is a way of organizing and presenting content on a website that emphasizes the meaning of the content rather than its appearance. By using semantic markup, web developers can create websites that are more accessible, easier to maintain, and more search engine friendly.

Web-Technology and Applications (Module1)

For example, instead of using a <div> tag to create a block of text, a semantic approach would use a <p> tag to indicate that the content is a paragraph. This not only makes the code easier to read and understand, but it also helps search engines to better understand the content of the page. In summary, semantic markup is an essential aspect of modern web development. By focusing on the meaning of the content rather than its appearance, web developers can create websites that are more accessible, easier to maintain, and more search engine friendly.

Structure

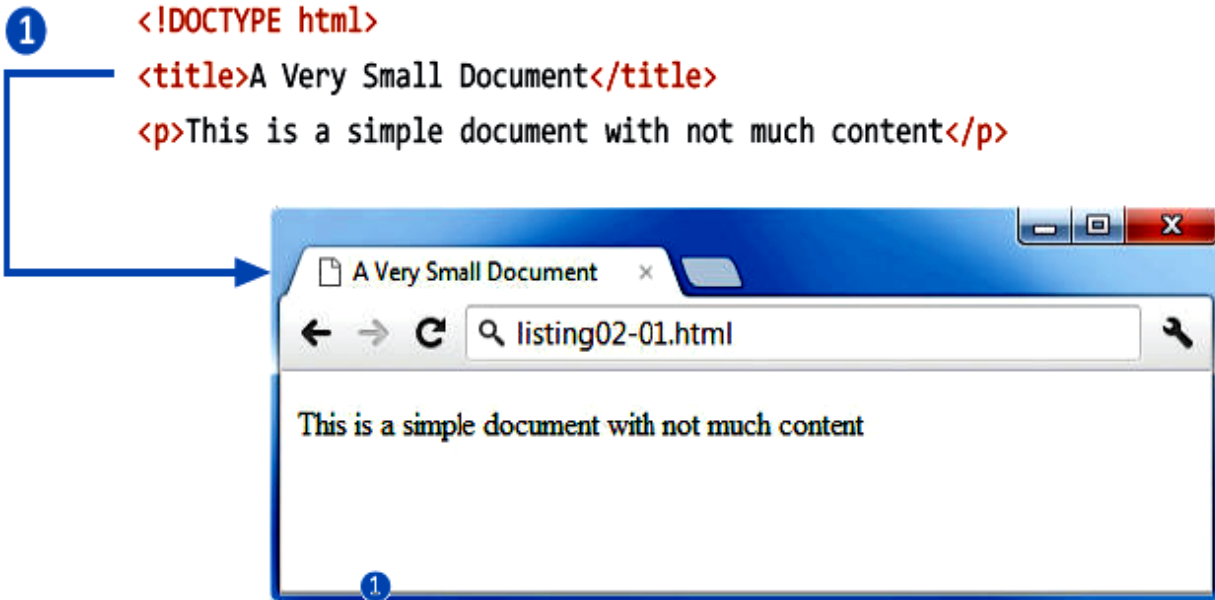
Structure is a vital way of communicating information in paper and electronic documents.

All of the tags that we will examine in this presentation are used to describe the basic structural information in a document, such as articles, headings, lists, paragraphs, links, images, navigation, footers, and so on.

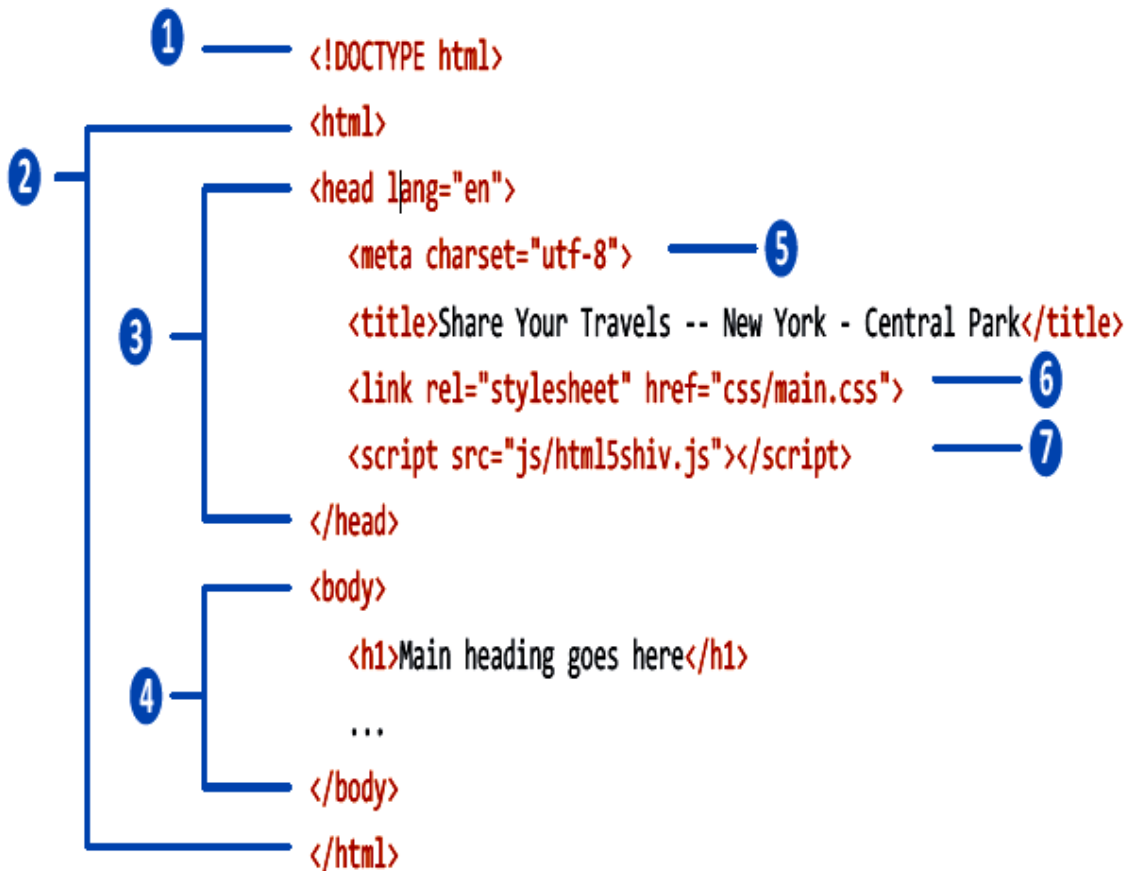
Eliminating presentation-oriented markup and instead writing semantic HTML markup offers a multitude of significant advantages.

Firstly, it enhances ***maintainability*** as semantic markup is easier to update and modify compared to web pages that contain a great deal of presentation markup. Secondly, it speeds up the process of authoring web pages and makes them ***faster*** to download. Thirdly, it improves ***accessibility*** for users who rely on voice reading software, as semantic markup provides a clear structure for the content. Lastly, it boosts ***search engine optimization*** by providing better instructions for search engines, allowing them to identify and prioritize important content on the site. By implementing semantic markup, websites can improve their overall functionality and user experience.

Structure of HTML



The `<title>` element, also known as the `Item`, serves as a crucial component in providing a concise and informative description of the content. Although the title is not visible within the browser window, it is prominently displayed by the browser in its window and/or tab. This element plays a vital role in enhancing the user experience by providing a clear and concise summary of the content, making it easier for users to navigate and understand the purpose of the webpage. Therefore, it is essential to ensure that the title accurately reflects the content of the page and is optimized for search engines to improve the visibility and accessibility of the webpage.



The **DOCTYPE**, which stands for Document Type Definition, is a crucial element in HTML coding. It informs the browser or any other client software that is reading the HTML document about the type of document it is about to process. It is important to note that the DOCTYPE does not indicate the version of HTML contained within the document. Instead, it only specifies that the document contains HTML.

By including the DOCTYPE in your HTML code, you ensure that the browser or client software can accurately interpret and display your document. This is especially important when working with different versions of HTML or when using advanced features that may not be supported by all browsers.

In summary, the DOCTYPE is a vital component of HTML coding that helps ensure your document is properly interpreted and displayed. By including it in your code, you can improve the overall functionality and accessibility of your website or application.

HTML, Head, and Body

HTML5 no longer mandates the use of the `<html>`, `<head>`, and `<body>` tags. However, in XHTML, these tags are still required, and most web authors continue to use them.

The `<html>` element is often referred to as the root element because it encompasses all other HTML elements in the document. It serves as the foundation for the entire web page and is responsible for defining the document type, character set, and language.

While HTML5 has made some changes to the way web pages are structured, understanding the importance of the `<html>`, `<head>`, and `<body>` tags is still crucial for web developers. These tags provide structure and organization to the web page, making it easier for search engines to crawl and index the content.

In summary, while HTML5 has relaxed some of the requirements for web page structure, the **`<html>`, `<head>`, and `<body>` tags** remain essential components for creating a well-structured and organized web page.

HTML pages are divided into two main sections: the head and the body, which correspond to the **`<head>` and `<body>`** elements. The head section contains descriptive elements that provide information about the document, such as the title, author, and keywords. On the other hand, the body section contains the actual content that will be displayed by the browser, such as text, images, videos, and interactive elements.

It is important to note that the head section is not visible to the user, but it is crucial for **search engine optimization (SEO)** and **accessibility purposes**. By including relevant and descriptive information in the head section, you can improve the visibility and ranking of your website in **search engine results pages (SERPs)**, as well as make it more accessible to users with disabilities who rely on assistive technologies.

In contrast, the body section is where the magic happens. This is where you can unleash your creativity and design skills to create visually appealing and engaging content that will capture the attention of your audience. Whether you are creating a blog post, a product page, or a landing page, the body section is where you can showcase your brand, your message, and your value proposition.

In summary, understanding the difference between the head and body sections of an HTML page is essential for creating effective and optimized web content. By mastering these two elements,

you can create websites that are both visually appealing and accessible, and that can help you achieve your business goals.

The <head> element is a crucial component of any web page, as it contains a variety of additional elements that help to optimize and enhance the user's experience. One of the most important of these is the <Meta> element, which declares the character encoding for the document. In our example, we have specified that the encoding is UTF-8, which ensures that the page will display correctly across a wide range of devices and platforms.

Another key element found within the <head> is the reference to an external CSS style sheet file. This file contains all of the styling information for the page, including fonts, colors, and layout. By separating this information from the HTML code, we can make our pages more modular and easier to maintain.

Finally, the <head> also references an external Javascript file. This file contains all of the interactive functionality for the page, such as dropdown menus, sliders, and pop-ups. By keeping this code separate from the HTML and CSS, we can ensure that our pages are fast-loading and responsive, even on slower connections.

In short, the <head> element is a vital part of any web page, and understanding its various components is essential for creating professional, engaging, and effective websites.

Quick tour of HTML

HTML5 is a robust language that offers a plethora of structural and presentation elements. However, it would be impractical to cover all of them in this presentation. Instead, we will provide a concise overview of the most commonly used elements. This approach will allow us to focus on the essential elements that are crucial for creating effective web pages. So, let's dive in and explore the key elements that will help you build stunning websites with HTML5.

```
<body>
1  <h1>Share Your Travels</h1>
   <h2>New York - Central Park</h2>
2  <p>Photo by Randy Connolly</p>
   <p>This photo of Conservatory Pond in
      <a href="http://www.centralpark.com/">Central Park</a> 3
      New York City was taken on October 22, 2011 with a
      <strong>Canon EOS 30D</strong> camera.
   </p> 4
5  

   <h3>Reviews</h3>
6  <div> 7
      <p>By Ricardo on <time>September 15, 2012</time></p>
      <p>Easy on the HDR buddy.</p>
   </div>

   <div>
      <p>By Susan on <time>October 1, 2012</time></p>
      <p>I love Central Park.</p>
   </div> 8
   <p><small>Copyright &copy; 2012 Share Your Travels</small></p>
</body> 9
```



Headings

Headings are a crucial aspect of document structure in HTML. With six levels of headings available (h1, h2, h3, and so on), authors can indicate the relative importance of each section. The higher the heading number, the less important the section.

By using headings, authors can guide their readers through the document's structure and make it easier to navigate. Headings also help to break up large blocks of text, making the document more visually appealing and easier to read.

In summary, headings are a powerful tool for document authors to create a clear and organized structure for their content. By using headings effectively, authors can improve the readability and accessibility of their documents, making them more engaging and informative for their readers.

The browser comes with pre-set styles for each heading level, but these can be easily adjusted and personalized using CSS. It is important to choose a heading level that accurately reflects the content's meaning, rather than selecting one based solely on its default appearance. For instance, opting for `<h3>` simply because it makes the text bold and 16pt is not a good practice. Instead, choose a heading level that is appropriate for the content, such as `<h3>` for a third-level heading rather than a primary or secondary heading. By doing so, the content will be more organized and easier to understand for readers.

Paragraphs

Paragraphs are the fundamental building blocks of any HTML document. They are denoted by the `<p>` tag, which serves as a container for text and other inline HTML elements. It is important to note that inline HTML elements do not cause a paragraph break, but rather blend seamlessly into the text's natural flow. By using paragraphs effectively, you can create a well-structured and easy-to-read document that is both professional and engaging.

Divisions:

The `<div>` tag serves as a container element that allows for the creation of logical groupings of content. Unlike other HTML tags, the `<div>` element has no intrinsic presentation, making it a versatile tool for web designers.

In modern CSS-based layouts, the `<div>` tag is frequently used to mark out sections, providing a clear visual hierarchy to the content.

Web-Technology and Applications (Module1)

While the use of <div> tags has been a staple of web design for years, HTML5 introduces a variety of new semantic elements that can help reduce the confusing mass of nested <div> tags that have become so typical in contemporary web design. We will explore these new elements in more detail later on.

Links

Links are established through the use of the <a> element, with the "a" representing anchor. A link is comprised of two primary components: the destination and the label. It is essential to ensure that both elements are properly configured to ensure that the link functions as intended. By utilizing the <a> element, web developers can create a seamless user experience that allows visitors to navigate through a website with ease.

```
<a href="http://www.centralpark.com">Central Park</a>
```



```
<a href="index.html"></a>
```



Types of Links:

The anchor element is a versatile tool that can be used to create a variety of links. Here are some examples:

1. Links to external sites: These links can direct users to other websites or specific resources, such as images or videos, on those sites.
2. Links to other pages or resources within the current site: These links can help users navigate to other pages or resources within the same website.
3. Links to other places within the current page: These links can be used to direct users to specific sections or elements within the same page.

4. Links to particular locations on another page: These links can be used to direct users to specific sections or elements on other pages.

5. Links that instruct the browser to start the user's email program: These links can be used to create email links that automatically open the user's email program with a pre-filled email address.

6. Links that instruct the browser to execute a Javascript function: These links can be used to trigger specific actions or functions within a website using Javascript.

By understanding the different types of links that can be created using the anchor element, web developers can create more effective and user-friendly websites.

Different link destinations

`<a href="http://www.centralpark.com/logo.gif">Central Park</a>`

Link to resource on external site

`<a href="index.html">Home</a>`

Link to another page on same site as this page

`<a href="#top">Go to Top of Document</a>`

Link to another place on the same page

`<a href="http://www.centralpark.com">Central Park</a>`

Link to external site

`<a href="productX.html#reviews">Reviews for product X</a>`

Link to specific place on another page

`<a href="mailto://person@somewhere.com">Someone</a>`

Link to email

`<a href="javascript://OpenAnnoyingPopup();">See This</a>`

Link to javascript function

`<a href="tel:+18009220579">Call toll free (800) 922-0579</a>`

Link to telephone (automatically dials the number when user clicks on it using a Smartphone browser).

Link Text

In the early days of the internet, links labeled with the generic phrase "Click Here" were ubiquitous. However, times have changed, and such links are now considered poor practice.

Why? For starters, they fail to inform users where the link will lead them. Additionally, the verb "click" is no longer an accurate descriptor, given the rise of mobile browsing.

To ensure a positive user experience, it's essential to use descriptive link labels. For example, instead of "Click here," try "See the race results" or simply "Race Results." By doing so, you'll provide users with a clear understanding of what they can expect when they click the link. This small change can make a big difference in the overall usability and accessibility of your website.

Absolute Referencing for URLs

When referring to a page or resource on an external website, it is essential to use a full absolute reference. This reference should include the following

1. The protocol, which is typically http://,
2. The domain name,
3. Any paths, and finally,
4. The file name of the desired resource.

By using absolute referencing, you ensure that your links are accurate and functional. This is particularly important when linking to external resources, as any errors in the URL can result in broken links and a poor user experience.

To avoid these issues, take the time to carefully construct your URLs using absolute referencing. This will help to ensure that your links are reliable, professional, and easy to use.

URL Relative Referencing

In order to effectively reference files within our website, it is crucial to understand the syntax for relative referencing. This technique allows us to reference resources that are located on the same server as our HTML document in a concise and efficient manner.

When referencing a resource on the same server, we can use relative referencing to avoid the need for a lengthy URL. It is important to note that if the URL does not include "http://", the browser will automatically request the file from the current server.

By mastering the art of relative referencing, we can streamline our website's functionality and improve the user experience. So, let's dive in and learn this essential skill!

Pathnames

Pathnames on the web adhere to the Unix conventions, which provide a standardized approach to organizing and accessing files and directories. To ensure clarity and consistency, it is important to understand the following guidelines:

- Directory names and file names are separated by forward slashes ("/").
- To reference a directory that is located above the current one in the directory tree, double-periods ("...") are used.

By following these conventions, users can easily navigate through the directory structure and locate the files they need. It is essential to maintain proper formatting and syntax to avoid confusion and errors.

Inline Text Elements

Inline elements do not disrupt the flow of text (i.e., cause a line break).HTML5 defines over 30 of these elements.

Element	Description
<a>	Anchor used for hyperlinks.
<abbr>	An abbreviation

	Line break
<cite>	Citation (i.e., a reference to another work).
<code>	Used for displaying code, such as markup or programming code.
	Emphasis
<mark>	For displaying highlighted text
<small>	For displaying the fine-print, i.e., "non-vital" text, such as copyright or legal notices.
	The inline equivalent of the <div> element. It is generally used to mark text that will receive special formatting using CSS.
	For content that is strongly important.
<time>	For displaying time and date data

Web-Technology and Applications (Module1)

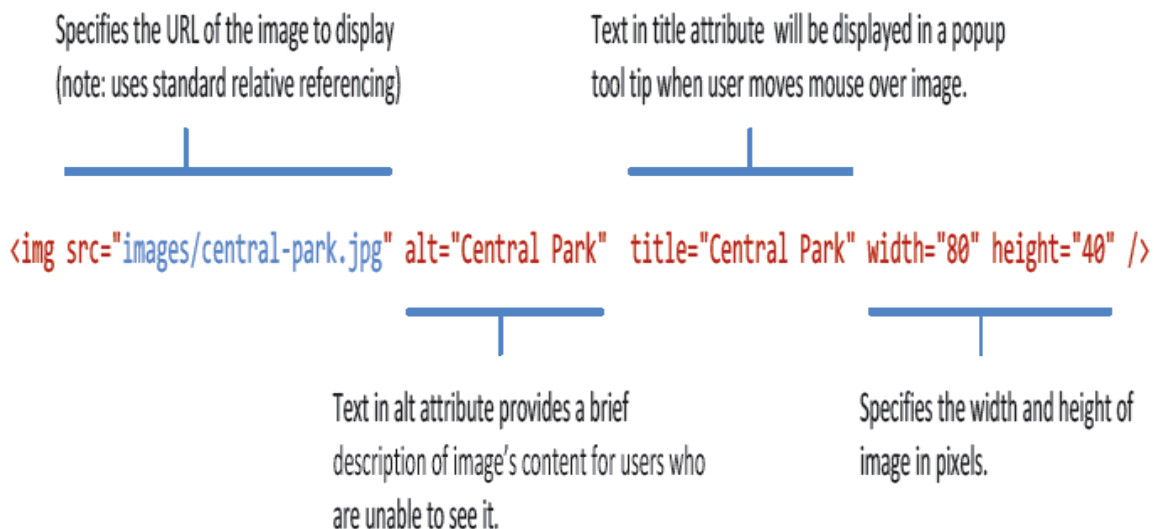
Images

Images are an essential component of web design, and while the `<img>` tag is the oldest method for displaying an image, it is not the only way. For purely decorative images, such as background gradients, patterns, logos, and border art, it makes semantic sense to keep them out of the markup and in CSS where they more rightly belong. However, when the images are content, such as in a gallery or the image of a product in a product details page, then the `<img>` tag is the semantically appropriate approach.

It is important to note that the `<img>` tag is not just a simple tool for displaying images. It also has several attributes that can be used to optimize the image for web use. These attributes include the "alt" attribute, which provides alternative text for the image in case it cannot be displayed, and the "title" attribute, which provides additional information about the image when the user hovers over it.

Furthermore, it is crucial to ensure that images are optimized for web use to improve website performance. This can be achieved by compressing the image file size, using the appropriate file format, and resizing the image to the appropriate dimensions.

In conclusion, while the `<img>` tag is not the only way to display images, it is still a crucial tool for displaying content images on a website. By optimizing images for web use and using the appropriate attributes, web designers can ensure that their images are not only visually appealing but also accessible and efficient.



Web-Technology and Applications (Module1)

Lists

Lists are a fundamental element of web design that allow for the organization and presentation of information in a clear and concise manner. There are three main types of lists: unordered, ordered, and definition lists.

Unordered lists are collections of items that are not arranged in any particular order. These lists are typically displayed as bulleted lists by default in web browsers.

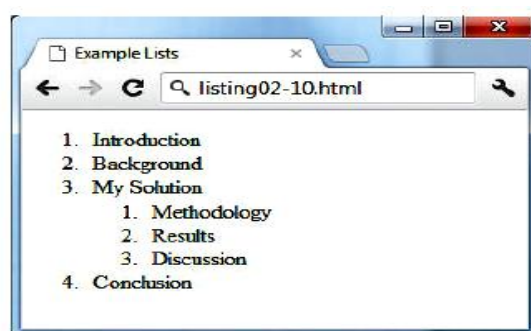
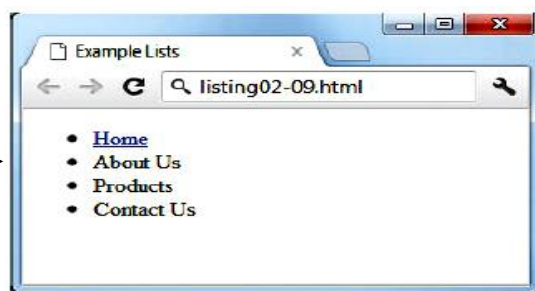
On the other hand, ordered lists are collections of items that have a specific order or sequence. These lists are usually displayed as numbered lists by default in web browsers.

Lastly, definition lists are collections of name and definition pairs. Although they are not commonly used, they can be particularly useful for presenting information in a clear and concise manner. A common example of a definition list is a FAQ list.

By utilizing lists in web design, you can effectively organize and present information to your audience in a way that is both visually appealing and easy to understand.

<pre> Home About Us Products Contact Us </pre>	<pre> Introduction Background My Solution Methodology Results Discussion Conclusion </pre>
--	--

Notice that the list item element can contain other HTML elements.



Character Entities

Character Entities are special symbols used in HTML that cannot be easily typed using a keyboard or have a reserved meaning in HTML. Examples of such symbols include accented characters and the copyright symbol. To use these symbols in an HTML document, you can either use the entity name or the entity number. For instance, you can use ` ` for a non-breaking space and `©` for the copyright symbol. By using character entities, you can enhance the visual appeal and clarity of your HTML documents.

Entity Name	Entity Number	Description
<code>&nbsp;</code>	<code>&#160;</code>	Nonbreakable space. The browser ignores multiple spaces in the source HTML file. If you need to display multiple spaces, you can do so using the nonbreakable space entity.
<code>&lt;</code>	<code>&#60;</code>	Less than symbol (" <code><</code> ").
<code>&gt;</code>	<code>&#62;</code>	Greater than symbol (" <code>></code> ").
<code>&copy;</code>	<code>&#169;</code>	The © copyright symbol.
<code>&euro;</code>	<code>&#8364;</code>	The € euro symbol.
<code>&trade;</code>	<code>&#8482;</code>	The ™ trademark symbol.
<code>&uuml;</code>	<code>&#252;</code>	The ü—i.e., small u with umlaut mark.

TABLE 2.3 Common Character Entities

HTML semantic elements

Header and Footer:

The majority of web pages have a distinct header and footer section that is easily recognizable. The header typically includes the site's logo, title, and possibly additional subtitles or taglines. Additionally, it may contain horizontal navigation links and one or two horizontal banners.

The footer, on the other hand, is located at the bottom of the page and often includes important information such as copyright notices, contact information, and links to social media accounts. It may also contain additional navigation links or a site map.

Having a well-designed header and footer is crucial for any website as it provides a consistent and professional look throughout the site. It also helps users navigate the site more easily and find the information they need quickly.

Web-Technology and Applications (Module1)

In summary, a well-crafted header and footer are essential components of any website. They not only provide a professional appearance but also aid in navigation and user experience.

A website's footer typically contains less critical information, such as smaller text versions of the navigation, copyright notices, details about the site's privacy policy, and potentially links to other social media platforms like Twitter. These elements are often located at the bottom of a webpage and serve as a convenient reference point for users seeking additional information about the site. While they may not be the most prominent features, they are nonetheless essential components of a well-designed website.

The HTML5 <header> and <footer> elements are not limited to serving as page headers and footers. They can also function as header and footer elements within other HTML5 containers, such as <article> or <section>. This versatility allows for greater flexibility in designing and structuring web pages. By utilizing these elements in various contexts, web developers can create more cohesive and organized layouts.

<header>

<h1>Fundamentals of Web Development</h1>

...

</header>

<article>

<header>

<h2>HTML5 Semantic Structure Elements

</h2>

<p>By Randy Connolly</p>

<p><time>September 30, 2012</time></p>

</header>

Heading Groups

The <hgroup> element can be used to group related headings together within one container.

<hgroup>

<header>

<hgroup>

<h1>Chapter Two: HTML 1</h1>

```
<h2>An Introduction</h2>
```

```
</hgroup>
```

```
</header>
```

```
<article>
```

```
<hgroup>
```

```
<h2>HTML5 Semantic Structure Elements </h2>
```

```
<h3>Overview</h3>
```

```
</hgroup>
```

```
</article>
```

Navigation

Navigation is an essential aspect of any website, and the <nav> element is a crucial tool for creating effective navigation systems. This element represents a section of a page that contains links to other pages or other parts within the same page. It is one of the new HTML5 semantic elements that do not receive any special presentation from the browser.

The <nav> element is specifically designed for major navigation blocks, such as global and secondary navigation systems. It is an excellent way to organize and structure your website's content, making it easier for users to find what they are looking for. By using this element, you can create a clear and concise navigation system that enhances the user experience and improves the overall functionality of your website.

In conclusion, the <nav> element is a powerful tool that can help you create effective navigation systems for your website. By using this element, you can organize your content and make it easier for users to find what they need. So, if you want to improve your website's navigation, be sure to incorporate the <nav> element into your design.

```
<header>
```

```

```

```
<h1>Fundamentals of Web Development</h1>
```

```
<nav role="navigation">
```

```
<ul>
```

```
<li><a href="index.html">Home</a></li>
```

```
<li><a href="about.html">About Us</a></li>
```

```
<li><a href="browse.html">Browse</a></li>
```


</nav>

</header>

Articles and Sections

The <article> element is used to represent a distinct section of content that stands alone within a document or website. This could include a magazine or newspaper article, or a blog post. On the other hand, the <section> element is used to represent a section of a document that typically has a title or heading.

It's important to note that according to the W3C, the <section> element is much broader in its application, while the <article> element is specifically intended for blocks of content that can be read or consumed independently of the other content on the page.

By using these elements correctly, you can ensure that your content is organized in a clear and logical manner, making it easier for users to navigate and understand. So whether you're creating a blog post or a complex website, be sure to use the appropriate elements to structure your content effectively.

Sections vs. Divs

The WHATWG specification cautions readers that the <section> element is not a catch-all container element. HTML already has the <div> element for that purpose. If an element is only needed for styling or scripting convenience, it's best to use the <div> element.

To determine whether to use the <section> element, consider whether the element's contents should be explicitly listed in the document's outline. If so, then use a <section>; otherwise, use a <div>.

By following these guidelines, you can ensure that your HTML code is both efficient and effective.

Figure and Figure Captions

According to the W3C Recommendation, the <figure> element is not limited to images alone. It can be used for any essential content that can be moved to a different location in the page or document without affecting the coherence of the rest of the document.

Web-Technology and Applications (Module1)

However, it is important to note that the `<figure>` element should not be used to wrap every image. For instance, nonessential images such as banner ads, site logos, and graphical embellishments should not be wrapped within `<figure>` elements.

Instead, the `<figure>` element should only be used for circumstances where the image (or other content) has a caption and where the figure is essential to the content but its position on the page is relatively unimportant.

By following these guidelines, you can ensure that your use of the `<figure>` element is both effective and appropriate.

```
<p>This photo was taken on October 22, 2011 with a Canon EOS 30D camera.</p>
```

```
<figure>
```

```
<br/>
```

```
<figcaption>Conservatory Pond in Central Park</figcaption>
```

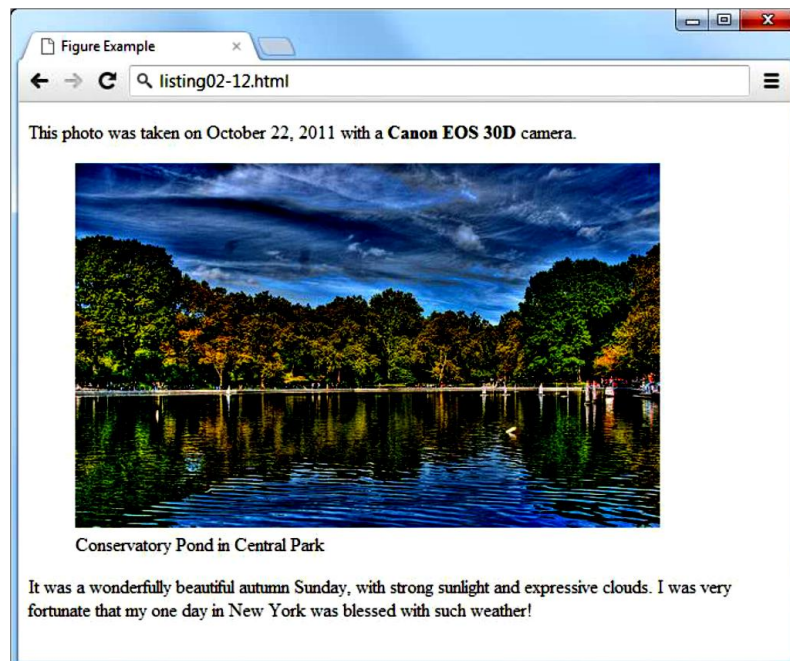
```
</figure>
```

```
<p>
```

It was a wonderfully beautiful autumn Sunday, with strong sunlight and expressive clouds. I was very fortunate that my one day in New York was blessed with such weather!

```
</p>
```

Figure could be moved to a different location in document ... But it has to exist in the document (i.e., the figure isn't optional).



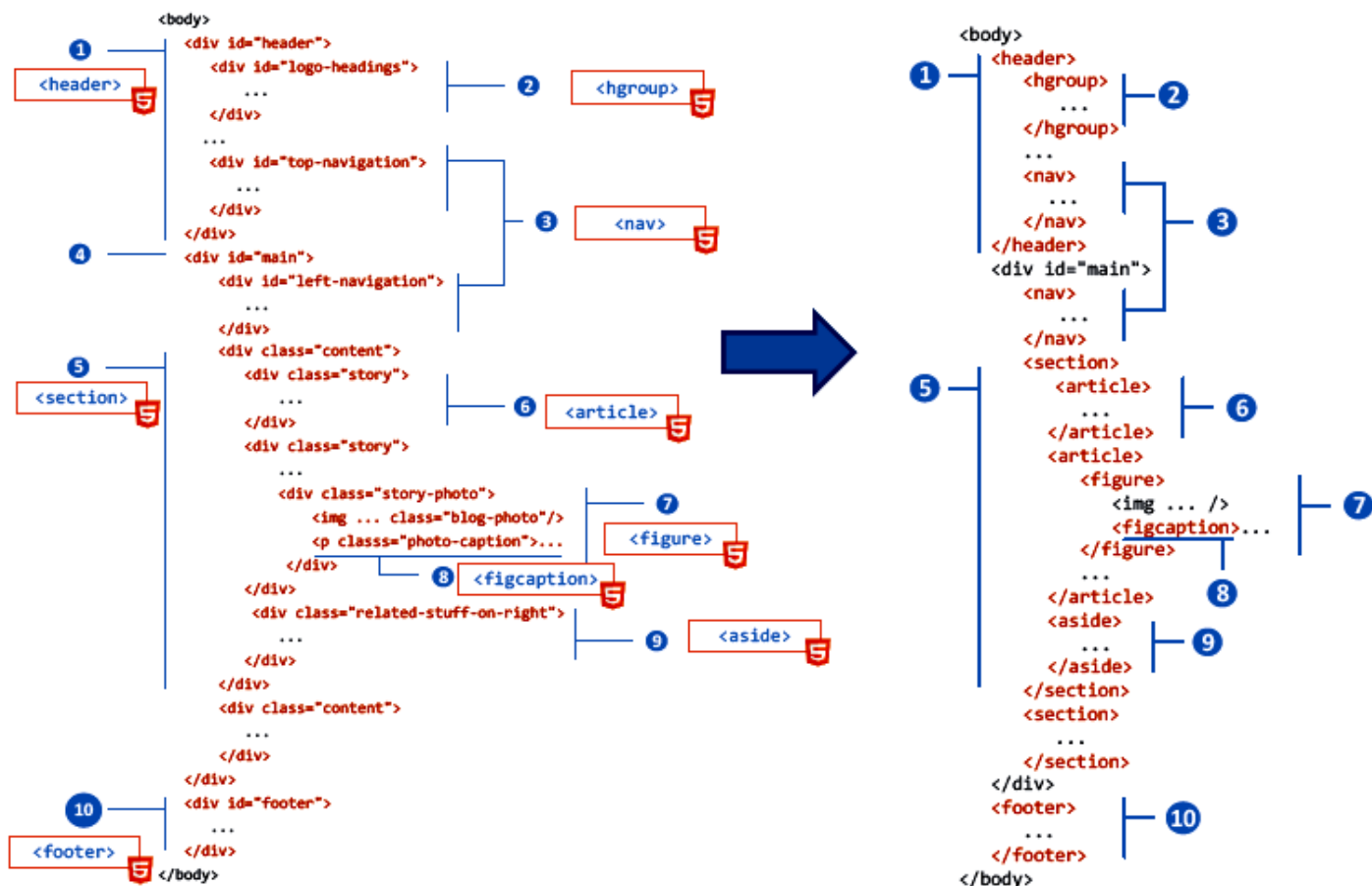
Aside

The `<aside>` element is a useful tool for web developers who want to separate certain content from the main body of a webpage. It is similar to the `<figure>` element, but with a slightly different purpose. While `<figure>` is used for important information that can be placed anywhere on the page, `<aside>` is specifically for content that is only tangentially related to the surrounding text.

This versatile element can be used for a variety of purposes, such as sidebars, pull quotes, or groups of advertising images. Essentially, any non-essential content can be grouped together using `<aside>`.

So, if you're looking to add some extra information to your webpage without cluttering up the main body of text, consider using the `<aside>` element. It's a simple and effective way to keep your content organized and easy to read.

XHTML versus HTML5



CSS

What is CSS?

CSS is a widely recognized standard developed by the **World Wide Web Consortium (W3C)** that enables the description of the presentation or appearance of HTML elements. This powerful tool allows developers to assign a variety of properties to their web pages, including font styles, colors, sizes, borders, background images, and even the positioning of elements. By utilizing CSS, developers can create visually appealing and professional-looking web pages that are both functional and aesthetically pleasing. With its versatility and ease of use, CSS has become an essential component of modern web development.

CSS, or Cascading Style Sheets, is a widely recognized standard developed by the **World Wide Web Consortium (W3C)** for defining the presentation or appearance of HTML elements. This powerful tool allows developers to assign various properties to HTML elements, including font styles, colors, sizes, borders, background images, and even the positioning of elements on a page. As a language, CSS has its own set of syntax rules that must be followed in order to achieve the desired results. It can be added directly to any HTML element using the style attribute, within the **<head>** element, or in a separate text file that contains only CSS.

By mastering CSS, you will be able to create visually stunning websites that are both professional and engaging. So why wait? Start exploring the endless possibilities of CSS today and take your web development skills to the next level!

Benefits of CSS:

CSS provides a higher degree of formatting control compared to HTML. This means that web designers can create more visually appealing and professional-looking websites with ease.

By centralizing all formatting into one or a small handful of CSS files, web developers can significantly improve the maintainability of their websites. This makes it easier to update and modify the design of a website without having to go through each individual HTML file.

CSS-driven sites are also more accessible, making them easier to navigate for users with disabilities. This is because CSS allows for the separation of content and presentation, which makes it easier for assistive technologies to interpret the content of a website.

Web-Technology and Applications (Module1)

In addition, websites built using a centralized set of CSS files for all presentation will also load faster. This is because each individual HTML file will contain less markup, resulting in quicker download times and a better user experience.

Finally, CSS can be used to adapt a page for different output mediums, such as print or mobile devices. This means that web designers can create a single website that is optimized for multiple devices, making it easier for users to access and navigate their content.

CSS Versions:

The World Wide Web Consortium (W3C) has published several versions of CSS over the years. The first version, CSS Level 1, was recommended in 1996. A year later, the more concise CSS Level 2 Recommendation, also known as CSS2, was released.

Despite work beginning over a decade ago, an updated version of the Level 2 Recommendation, CSS2.1, did not become an official W3C Recommendation until June 2011. This delay was due to the complexity of the standard and the need for extensive testing to ensure its compatibility with various web browsers.

To further complicate matters, a different group at the W3C was simultaneously working on a CSS3 draft during the last decade and continues to do so today. CSS3 promises to bring exciting new features to web design, including advanced layout options, animations, and transitions.

In summary, CSS has come a long way since its inception in 1996, and its evolution continues to shape the way we design and develop websites.

Browser Adoption:

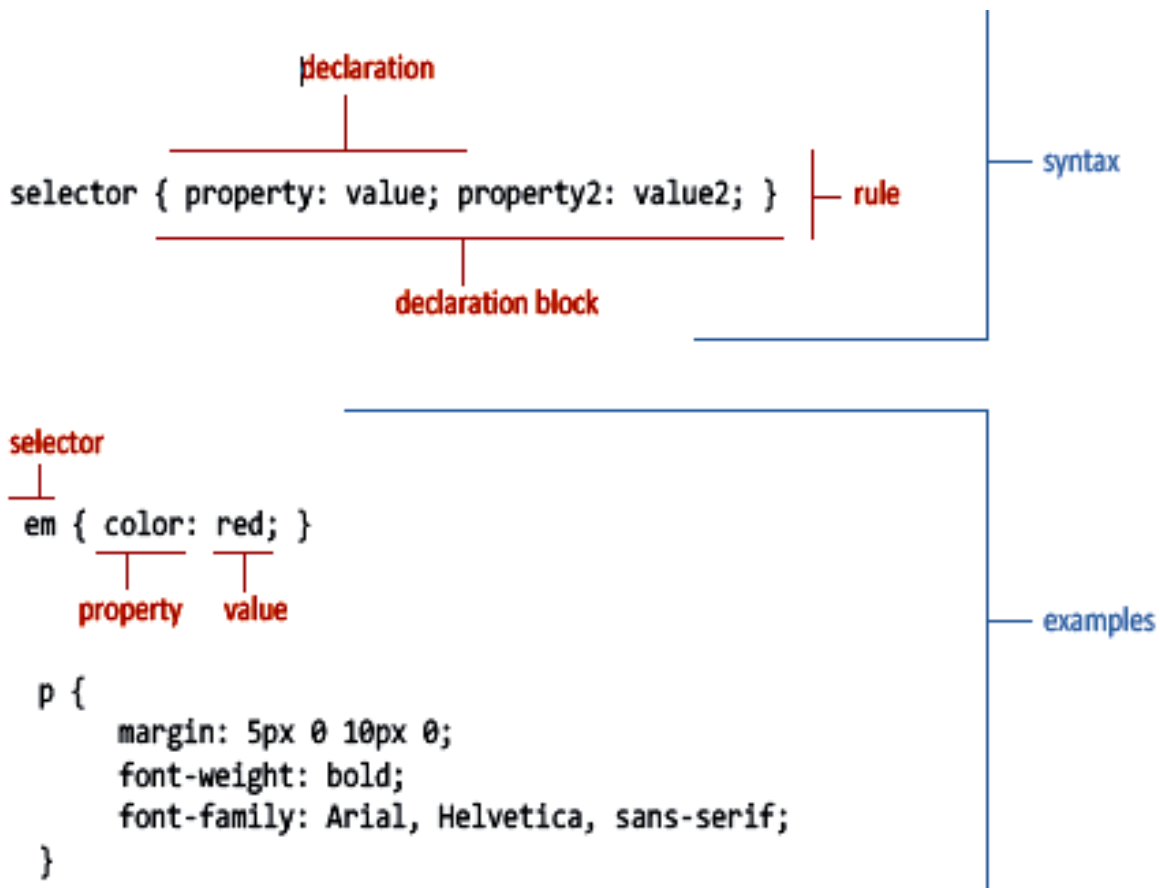
Although Microsoft's Internet Explorer was an early advocate for CSS, its later versions, particularly IE5, IE6, and IE7 for Windows, had inconsistent support for certain aspects of CSS2. However, it is important to note that all browsers have left certain parts of the CSS2 Recommendation unimplemented.

CSS has a reputation for being a challenging language to work with, and unfortunately, this reputation is well-deserved. Despite its challenges, CSS remains a crucial tool for web developers and designers alike. As such, it is important to stay up-to-date with the latest developments in browser adoption and CSS implementation. By doing so, we can ensure that our websites are both visually appealing and functional across a wide range of devices and platforms.

CSS syntax

A **Cascading Style Sheet (CSS)** document is composed of one or more style rules. Each rule comprises a selector that identifies the HTML element or elements that will be impacted, followed by a sequence of property and value pairs, also known as declarations.

CSS is a crucial component of web design, as it allows developers to control the visual appearance of a website. By using CSS, designers can specify the font, color, layout, and other stylistic elements of a webpage. Understanding the structure of a CSS document and the syntax of its rules is essential for creating effective and visually appealing websites.

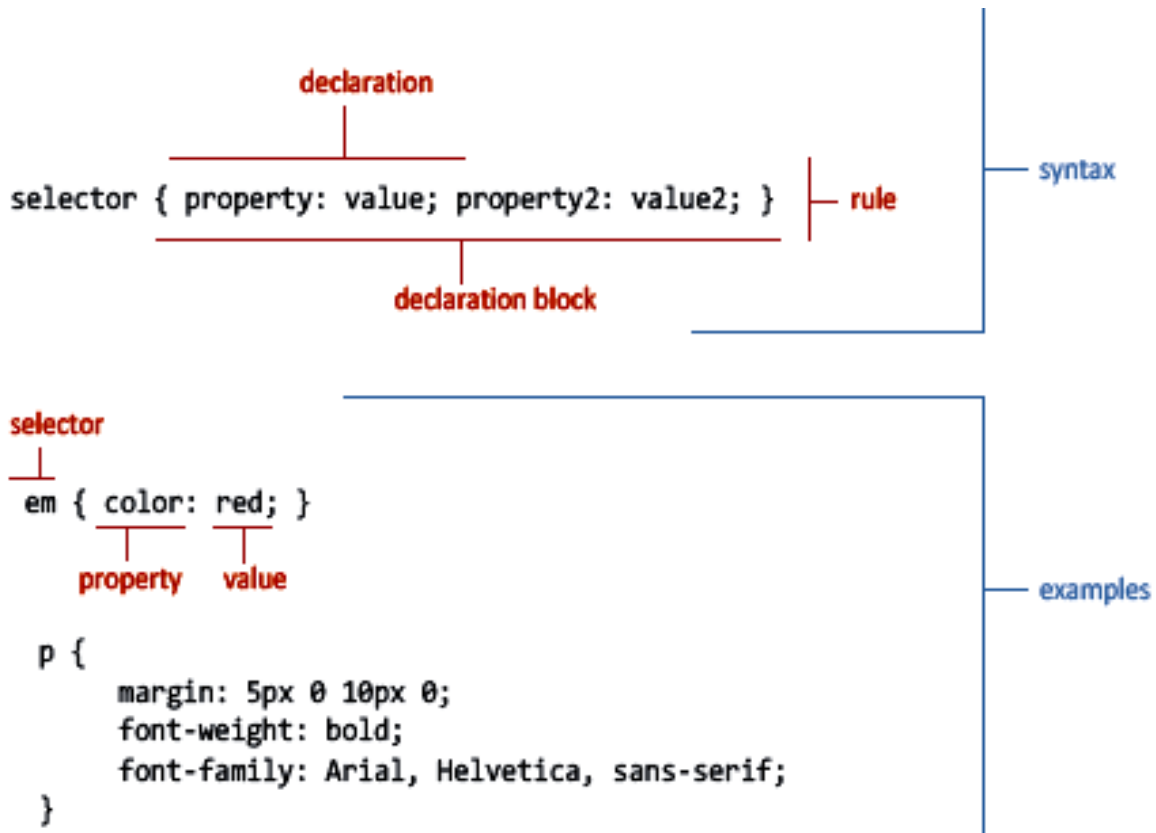


Declaration Blocks:

A group of declarations is commonly referred to as a declaration block. This block can either be written on a single line or spread out across multiple lines. It is important to note that the browser disregards any white space present within the block. Additionally, each declaration must be concluded with a semicolon.

Web-Technology and Applications (Module1)

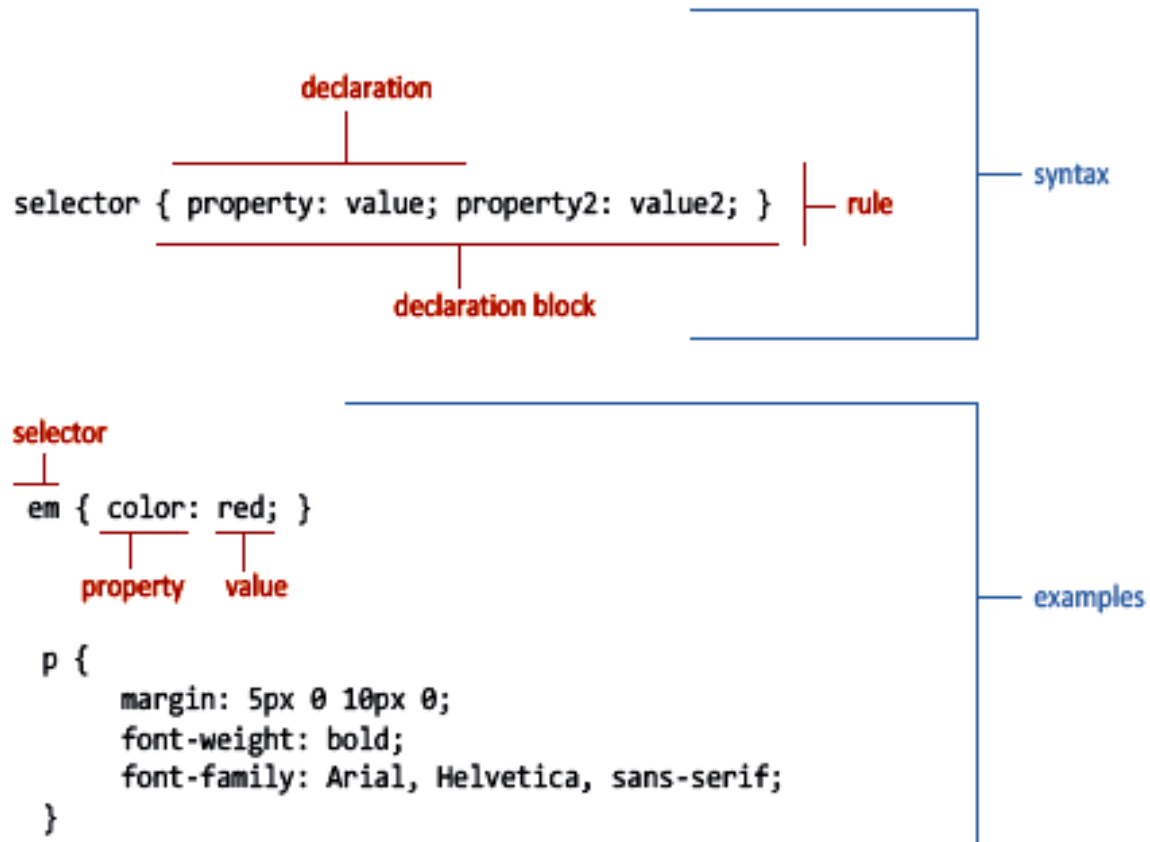
In summary, declaration blocks are a crucial aspect of coding and must be properly formatted to ensure the correct execution of code. By understanding the importance of declaration blocks, developers can create efficient and effective code that produces the desired results.



Selectors

Selectors are a fundamental aspect of CSS. They serve as the starting point for every CSS rule. Essentially, a selector is used to identify which HTML element(s) will be impacted by the declarations within a given rule.

Think of selectors as a pattern that the browser uses to select the specific HTML elements that will receive the desired style. By using selectors effectively, you can ensure that your CSS rules are applied only to the elements you intend to target, resulting in a more polished and professional-looking website.

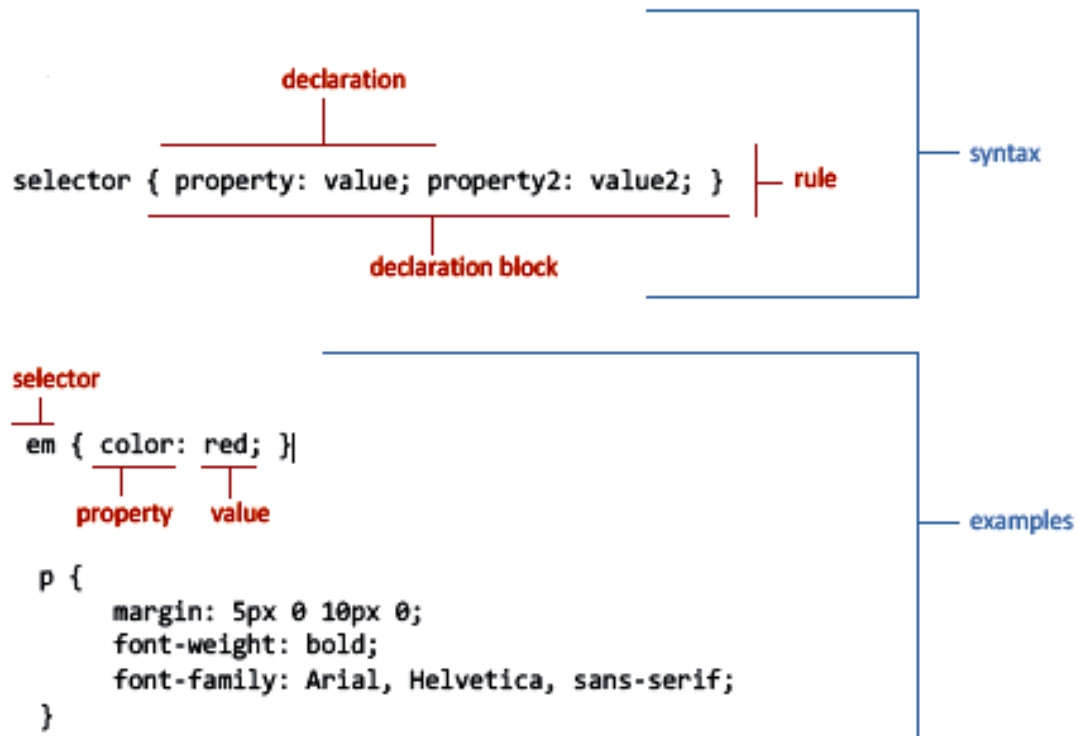


Properties:

- Each CSS declaration must contain at least one property.
- These property names are predetermined by the CSS standard.
- The CSS2.1 Recommendation outlines more than a hundred distinct property names.

CSS properties are essential components of any CSS declaration. They define the specific characteristics of an element's style, such as its color, size, and position. These property names are predetermined by the CSS standard, ensuring consistency and compatibility across different web browsers and devices.

The CSS2.1 Recommendation, a widely recognized standard for CSS, outlines over a hundred distinct property names. These properties cover a wide range of design elements, from basic font styles to complex layout and animation effects. By utilizing these properties effectively, web developers can create visually stunning and highly functional websites that engage and delight their users.



Property Type	Property
Fonts	font font-family font-size font-style font-weight @font-face
Text	letter-spacing line-height text-align text-decoration text-indent
Color and background	background background-color background-image background-position background-repeat color
Borders	border border-color border-width border-style border-top border-top-color border-top-width etc

Property Type	Property
Spacing	padding padding-bottom, padding-left, padding-right, padding-top margin margin-bottom, margin-left, margin-right, margin-top
Sizing	height max-height max-width min-height min-width width
Layout	bottom, left, right, top clear display float overflow position visibility z-index
Lists	list-style list-style-image list-style-type

Values

Values are an integral part of CSS declarations, as they determine the specific property that is being applied. It is important to note that the unit of a value is dependent upon the property being used. For instance, some properties have predefined lists of keywords, while others utilize values such as length measurements, percentages, and numbers without units, color values, and URLs. By understanding the various types of values that can be used in CSS, developers can create more dynamic and visually appealing web pages.

Method	Description	Example
Name	Use one of 17 standard color names. CSS3 has 140 standard names.	color: red; color: hotpink; /* CSS3 only */
RGB	Uses three different numbers between 0 and 255 to describe the Red, Green, and Blue values for the color.	color: rgb(255,0,0); color: rgb(255,105,180);
Hexadecimal	Uses a six-digit hexadecimal number to describe the red, green, and blue value of the color; each of the three RGB values is between 0 and FF (which is 255 in decimal). Notice that the hexadecimal number is preceded by a hash or pound symbol (#).	color: #FF0000; color: #FF69B4;
RGBA	Allows you to add an alpha, or transparency, value. This allows a background color or image to “show through” the color. Transparency is a value between 0.0 (fully transparent) and 1.0 (fully opaque).	color: rgb(255,0,0, 0.5);
HSL	Allows you to specify a color using Hue Saturation and Light values. This is available only in CSS3. HSLA is also available as well.	color: hsl(0,100%,100%); color: hsl(330,59%,100%);

Units of Measurement

When it comes to web design, understanding units of measurement is crucial. There are two types of units: relative and absolute. Relative units are based on the value of something else, such as the size of a parent element. On the other hand, absolute units have a real-world size.

It is recommended to avoid using absolute units unless you are defining a style sheet for printing. However, pixels are a popular exception to this rule. Although, as we will see later, there are also good reasons for avoiding the pixel unit.

In summary, knowing the difference between relative and absolute units is essential for creating a successful web design. By using relative units, you can ensure that your design is flexible and adaptable to different screen sizes. So, choose your units wisely and create a website that is both functional and visually appealing.

Unit	Description	Type
px	Pixel. In CSS2 this is a relative measure, while in CSS3 it is absolute (1/96 of an inch).	Relative (CSS2) Absolute (CSS3)
em	Equal to the computed value of the font-size property of the element on which it is used. When used for font sizes, the em unit is in relation to the font size of the parent.	Relative
%	A measure that is always relative to another value. The precise meaning of % varies depending upon which property it is being used.	Relative
ex	A rarely used relative measure that expresses size in relation to the x-height of an element's font.	Relative
ch	Another rarely used relative measure; this one expresses size in relation to the width of the zero ("0") character of an element's font.	Relative (CSS3 only)
rem	Stands for root em, which is the font size of the root element. Unlike em, which may be different for each element, the rem is constant throughout the document.	Relative (CSS3 only)
vw, vh	Stands for viewport width and viewport height. Both are percentage values (between 0 and 100) of the viewport (browser window). This allows an item to change size when the viewport is resized.	Relative (CSS3 only)

Table: Relative units

Unit	Description	Type
in	Inches	Absolute
cm	Centimeters	Absolute
mm	Millimeters	Absolute
pt	Points (equal to 1/72 of an inch)	Absolute
pc	Pica (equal to 1/6 of an inch)	Absolute

Table: Absolute units

Comments in CSS

Comments in CSS are a valuable tool for any web developer. They provide a way to add notes and explanations to your style sheets, making it easier to understand and maintain your code. To add a comment in CSS, simply enclose your text within the following syntax: **`/* comment goes here */`**.

By using comments, you can provide context for your code, explain your thought process, and make it easier for others to collaborate with you. Additionally, comments can help you troubleshoot issues and make changes more efficiently.

So, don't underestimate the power of comments in CSS. Take advantage of this simple yet effective tool to improve the quality and readability of your code.

Locations of styles

There are actually three distinct types of style sheets that are commonly used in web design. The first type is known as **author-created style sheets**, which is what we are currently learning about in this presentation. The second type is **user style sheets**, which allow individual users to customize the way that web pages are displayed in their browser. This option is typically found in the accessibility options area of most browsers. Finally, there is the **browser style sheet**, which defines the default styles that the browser uses for each HTML element. By understanding these different types of style sheets, web designers can create more customized and professional-looking websites that meet the needs of their users.

Locations for CSS Styles

CSS style rules can be located in three different places: inline, embedded, and external. These locations offer flexibility in how you apply styles to your web pages.

Inline styles are applied directly to an HTML element using the "style" attribute. This method is useful for making quick changes to a specific element, but can become cumbersome when making multiple changes across multiple pages.

Embedded styles are placed within the head section of an HTML document using the "style" tag. This method allows for more organization and control over styles, as multiple rules can be applied to multiple elements within the same document.

External styles are stored in a separate CSS file and linked to the HTML document using the "link" tag. This method is ideal for larger projects with multiple pages, as it allows for consistent styling across all pages and easier maintenance.

By combining all three locations, you can create a comprehensive and efficient system for applying styles to your web pages.

Inline Styles:

An inline style is a style that only affects the element it is defined within. It will override any other style definitions for the properties used in the inline style. However, using inline styles is generally discouraged since they increase bandwidth and decrease maintainability.

Despite this, inline styles can be handy for quickly testing out a style change. It is important to note that while inline styles may be useful in certain situations, it is best to use external style sheets for larger projects to ensure consistency and ease of maintenance.

```
<h1>Share Your Travels</h1>
<h2>style="font-size: 24pt"Description</h2>
...
<h2>style="font-size: 24pt; font-weight: bold;">Reviews</h2>
```

LISTING 3.1 Internal styles example

Embedded Style Sheet:

Although embedded styles are an improvement over inline styles, they are generally not recommended. Each HTML document has its own <style> element, making it more challenging to maintain consistent styling across multiple documents when using embedded styles.

To ensure a professional and cohesive appearance across your website, it is recommended to use an external style sheet. This allows for easy modification and application of styles to multiple pages, resulting in a more efficient and streamlined design process. By utilizing an external style sheet, you can also improve the loading speed of your website, as the style sheet can be cached by the browser for faster access.

In summary, while embedded styles may seem like a convenient option, it is best to avoid them in favor of an external style sheet for a more professional and efficient website design.

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <style>
    h1 { font-size: 24pt; }
    h2 {
      font-size: 18pt;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <h1>Share Your Travels</h1>
  <h2>New York - Central Park</h2>
  ...
```

LISTING 3.2 Embedded styles example

External Style Sheet:

The external style sheet is the most commonly used location for style rules, as it offers the best maintainability. When you make a change to an external style sheet, all HTML documents that reference it will automatically use the updated version. This saves time and effort, as you don't have to manually update each individual HTML document.

Additionally, the browser is able to cache the external style sheet, which can improve the performance of the site. This means that the browser can store a copy of the style sheet on the user's computer, so that it doesn't have to download it every time the user visits the site. This can lead to faster load times and a better user experience.

To use an external style sheet, simply place your style rules within a text file with the .css extension. This file can then be linked to your HTML documents using the <link> tag. By using external style sheets, you can keep your code organized and easy to maintain, while also improving the performance of your site.

```
<head lang="en">
  <meta charset="utf-8">
  <title>Share Your Travels -- New York - Central Park</title>
  <link rel="stylesheet" href="styles.css" />
</head>
```

LISTING 3.3 Referencing an external style sheet

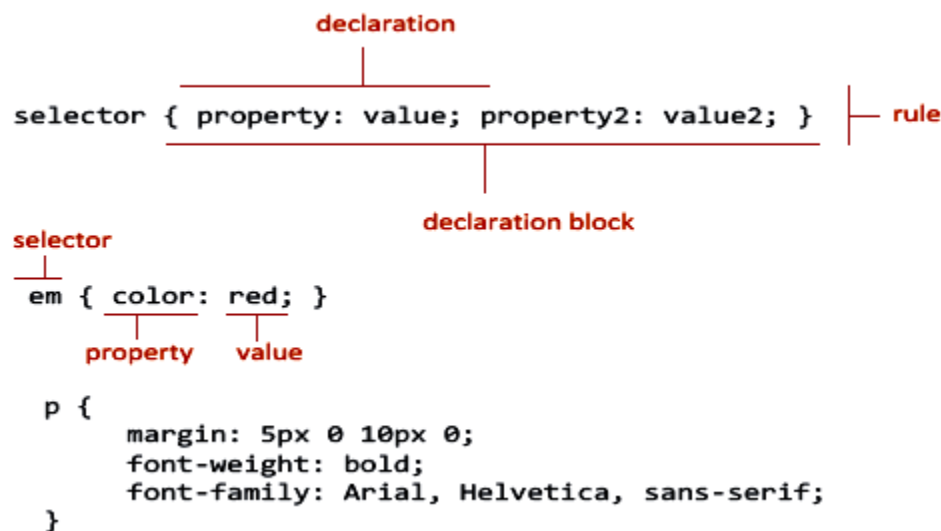
Selectors

Selectors are a crucial component when defining CSS rules. They serve the purpose of informing the browser which elements will be affected by the rules. CSS selectors offer a wide range of options, allowing you to select individual elements, multiple HTML elements, elements that belong together, or elements that are positioned in specific ways within the document hierarchy.

There are various types of selector options available to make your life easier. By utilizing these options, you can streamline the process of selecting the desired elements and applying the necessary rules. With a clear understanding of selectors, you can create visually appealing and professional-looking web pages that are easy to navigate and understand.

Element Selectors:

Element selectors utilize the HTML element name to target specific elements on a webpage. By using the universal element selector, denoted by the * (asterisk) character, you can select all elements on a page. This is a powerful tool for web developers and designers alike, allowing for precise control over the appearance and functionality of a website. With a solid understanding of element selectors, you can take your web development skills to the next level and create truly exceptional websites.



Grouped Selectors:

You can select a group of elements by separating the different element names with commas. This is a practical way to reduce the size and complexity of your CSS files by combining multiple

identical rules into a single rule. By selecting multiple elements at once, you can apply the same style to all of them simultaneously, saving time and effort.

For example, instead of writing separate rules for each heading element (h1, h2, h3, etc.), you can group them together like this:

```
h1, h2, h3 {  
    font-family: Arial, sans-serif;  
    color: #333;  
    text-align: center;  
}
```

This will apply the same font, color, and alignment to all heading elements, making your code more efficient and easier to read.

In summary, using grouped selectors is a smart way to streamline your CSS code and make it more manageable. By combining similar rules into a single selector, you can save time, reduce errors, and create a more professional-looking website.

```
/* commas allow you to group selectors */  
p, div, aside {  
    margin: 0;  
    padding: 0;  
}  
/* the above single grouped selector is equivalent to the  
   following: */  
p {  
    margin: 0;  
    padding: 0;  
}  
div {  
    margin: 0;  
    padding: 0;  
}  
aside {  
    margin: 0;  
    padding: 0;  
}
```

LISTING 3.4 Sample grouped selector

Reset:

Grouped selectors are commonly utilized to swiftly reset or eliminate browser defaults. The purpose of this action is to minimize browser inconsistencies regarding margins, line heights, and font sizes. These reset styles can be placed in a separate CSS file, aptly named reset.css, and linked to the page before any other external style sheets.

By implementing a reset, you can ensure that your website's design is consistent across all browsers and devices. This not only enhances the user experience but also presents a more professional and polished appearance. Don't let browser defaults hinder your website's potential - take control with a reset.

```
html, body, div, span, h1, h2, h3, h4, h5, h6, p {  
    margin: 0;  
    padding: 0;  
    border: 0;  
    font-size: 100%;  
    vertical-align: baseline;  
}
```

Class Selectors:

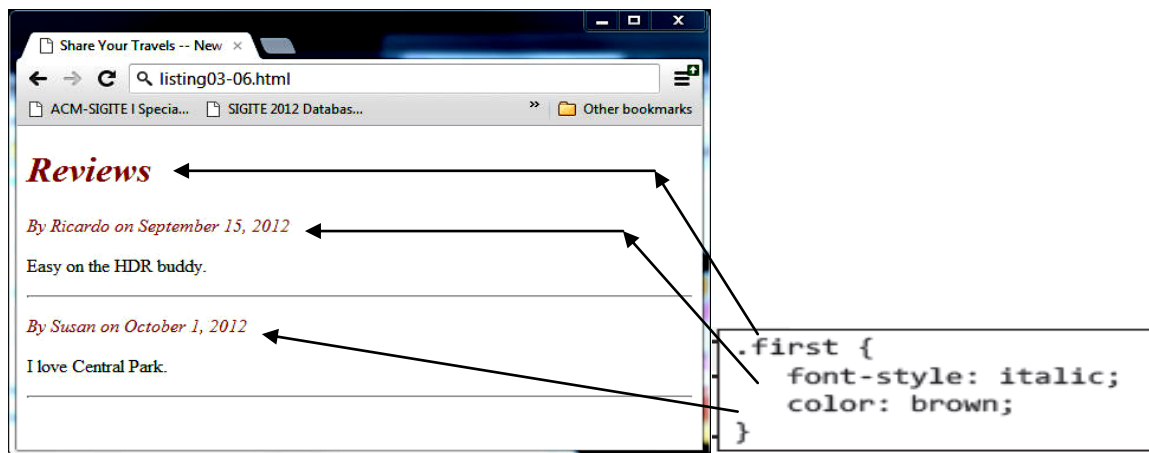
A class selector is a powerful tool that allows you to target multiple HTML elements at once, regardless of their position in the document tree. By labeling a series of HTML elements with the same class attribute value, you can easily apply styling to them using a class selector.

To use a class selector, simply use a period (.) followed by the class name. This will target all elements with that specific class name, making it easy to apply consistent styling throughout your website.

By utilizing class selectors, you can streamline your CSS code and make it easier to maintain and update. So why not give it a try and see how it can improve your website's design?

```
<head>  
<title>Share Your Travels </title>  
<style>  
.first {  
font-style: italic;  
color: brown;
```

```
}
</style>
</head>
<body>
<h1 class="first">Reviews</h1>
<div>
<p class="first">By Ricardo on <time>September 15, 2012</time></p>
<p>Easy on the HDR buddy.</p>
</div>
<hr/>
<div>
<p class="first">By Susan on <time>October 1, 2012</time></p>
<p>I love Central Park.</p>
</div>
<hr/>
</body>
```



ID Selectors

An ID selector is a powerful tool that allows you to target a specific element on a webpage by its unique ID attribute, regardless of its type or position. By using an ID selector, you can easily apply custom styles to a specific element, making it stand out from the rest of the page.

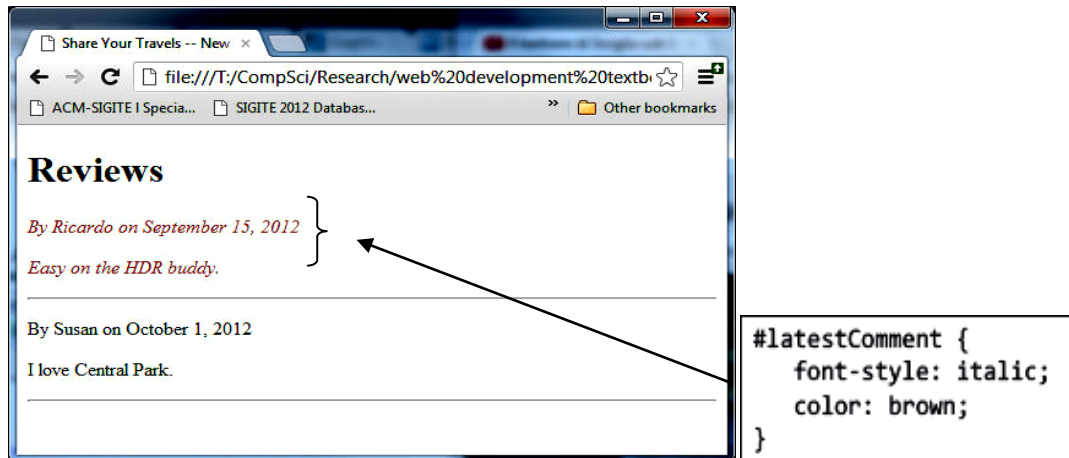
To use an ID selector, simply add an ID attribute to the HTML element you want to target, and then use the pound/hash (#) symbol followed by the ID name in your CSS code. It's important to

Web-Technology and Applications (Module1)

note that you should only use an ID once per page to avoid conflicts and ensure that your code is clean and efficient.

By mastering the use of ID selectors, you can take your web design skills to the next level and create truly unique and visually stunning websites. So why not give it a try and see what you can create?

```
<head lang="en">
<meta charset="utf-8">
<title>Share Your Travels -- New York - Central Park</title>
<style>
#latestComment {
font-style: italic;
color: brown;
}
</style>
</head>
<body>
<h1>Reviews</h1>
<div id="latestComment">
<p>By Ricardo on <time>September 15, 2012</time></p>
<p>Easy on the HDR buddy.</p>
</div>
<hr/>
<div>
<p>By Susan on <time>October 1, 2012</time></p>
<p>I love Central Park.</p>
</div>
<hr/>
</body>
```



Id selectors v/s class selectors

When it comes to selecting HTML elements, it's important to understand the difference between id and class selectors. Id selectors should only be used when referring to a single HTML element, as the id attribute can only be assigned to one element. On the other hand, class selectors are more appropriate when potentially referencing multiple related elements. By using class selectors, you can easily apply the same styles to all elements with the same class, streamlining your code and making it more efficient. So, next time you're selecting elements in your HTML, remember to choose the appropriate selector for the job!

Attribute Selectors:

An attribute selector is a powerful tool that allows you to select HTML elements based on the presence or value of an attribute. This technique can be incredibly useful in styling hyperlinks and images, among other things.

However, it's worth noting that not all web authors have taken advantage of attribute selectors due to uneven support from some browsers. Despite this, attribute selectors remain a valuable tool for those looking to create professional and visually appealing websites.

By using attribute selectors, you can easily target specific elements and apply styles that enhance the overall look and feel of your website. Whether you're a seasoned web developer or just starting out, attribute selectors are a technique worth exploring.

```
<head lang="en">
```

```
<meta charset="utf-8">
```

```
<title>Share Your Travels</title>
```

```
<style>
```

```
[title] {  
  cursor: help;  
  padding-bottom: 3px;  
  border-bottom: 2px dotted blue;  
  text-decoration: none;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div>
```

```

```

```
<h2><a href="countries.php?id=CA" title="see posts from Canada">
```

```
Canada</a>
```

```
</h2>
```

```
<p>Canada is a North American country consisting of ... </p>
```

```
<div>
```

```

```

```

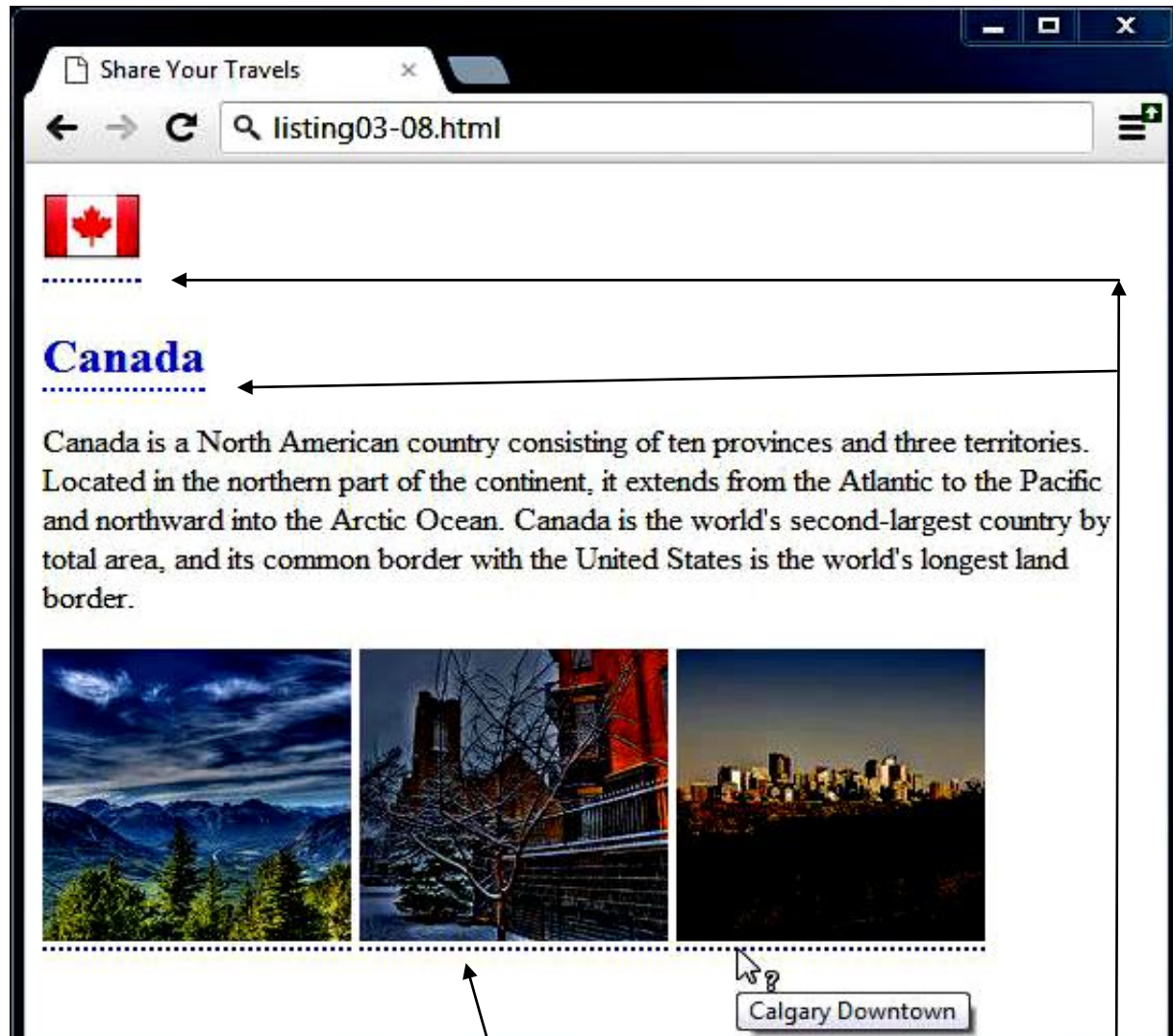
```

```

```

```
</div>
```

```
</div>
```



Share Your Travels x

listing03-08.html



Canada

Canada is a North American country consisting of ten provinces and three territories. Located in the northern part of the continent, it extends from the Atlantic to the Pacific and northward into the Arctic Ocean. Canada is the world's second-largest country by total area, and its common border with the United States is the world's longest land border.



Calgary Downtown

```
[title] {  
  cursor: help;  
  padding-bottom: 3px;  
  border-bottom: 2px dotted blue;  
  text-decoration: none;  
}
```

Selector	Matches	Example
[]	A specific attribute.	[title] Matches any element with a title attribute
[=]	A specific attribute with a specific value.	a[title="posts from this country"] Matches any <a> element whose title attribute is exactly "posts from this country"
[-=]	A specific attribute whose value matches at least one of the words in a space-delimited list of words.	[title~="Countries"] Matches any title attribute that contains the word "Countries"
[^=]	A specific attribute whose value begins with a specified value.	a[href^="mailto"] Matches any <a> element whose href attribute begins with "mailto"
[*=]	A specific attribute whose value contains a substring.	img[src*="flag"] Matches any element whose src attribute contains somewhere within it the text "flag"
[\$=]	A specific attribute whose value ends with a specified value.	a[href\$=".pdf"] Matches any <a> element whose href attribute ends with the text ".pdf"

Pseudo Selectors

Pseudo Selectors are a powerful tool in CSS that allow you to select elements that may not exist explicitly in the HTML document tree. These selectors come in two forms: pseudo-element selectors and pseudo-class selectors.

A pseudo-element selector is used to select an element that is not explicitly defined in the HTML document tree, but is still a recognizable selectable object. This can be useful for selecting elements such as the first letter of a paragraph or the content generated by the `::before` and `::after` pseudo-elements.

On the other hand, a pseudo-class selector applies to an HTML element and targets a particular state or family relationship. In CSS3, there are a variety of family relationships that can be targeted, such as the first child or last-of-type.

One of the most common uses of pseudo selectors is for targeting link states. For example, you can use the `:hover` pseudo-class selector to change the style of a link when the user hovers over it with their mouse.

By utilizing pseudo selectors in your CSS, you can create more dynamic and engaging web pages that are both visually appealing and easy to navigate.

```
<head>
  <title>Share Your Travels</title>
  <style>
    a:link {
      text-decoration: underline;
      color: blue;
    }
    a:visited {
      text-decoration: underline;
      color: purple;
    }
    a:hover {
      text-decoration: none;
      font-weight: bold;
    }
    a:active {
      background-color: yellow;
    }
  </style>
</head>
<body>
  <p>Links are an important part of any web page. To learn more about
    links visit the <a href="#">W3C</a> website.</p>
  <nav>
    <ul>
      <li><a href="#">Canada</a></li>
      <li><a href="#">Germany</a></li>
      <li><a href="#">United States</a></li>
    </ul>
  </nav>
</body>
```

LISTING 3.8 Styling a link using pseudo-class selectors

Contextual Selectors:

CSS3 introduces contextual selectors, also known as combinatory, which enable you to select elements based on their relationship to other elements in the document tree. These selectors allow you to target elements based on their ancestors, descendants, or siblings, making it easier to apply specific styles to specific parts of your web page.

Web-Technology and Applications (Module1)

By using contextual selectors, you can create more complex and dynamic styles that respond to the structure and content of your HTML document. For example, you could use a descendant selector to target all paragraphs within a specific div, or a sibling selector to style the first list item differently from the rest.

Overall, contextual selectors are a powerful tool for web developers and designers, allowing them to create more sophisticated and customized styles that enhance the user experience.

Selector	Matches	Example
Descendant	A specified element that is contained somewhere within another specified element	<code>div p</code> Selects a <code><p></code> element that is contained somewhere within a <code><div></code> element. That is, the <code><p></code> can be any descendant, not just a child.
Child	A specified element that is a direct child of the specified element	<code>div>h2</code> Selects an <code><h2></code> element that is a child of a <code><div></code> element.
Adjacent Sibling	A specified element that is the next sibling (i.e., comes directly after) of the specified element.	<code>h3+p</code> Selects the first <code><p></code> after any <code><h3></code> .
General Sibling	A specified element that shares the same parent as the specified element.	<code>h3~p</code> Selects all the <code><p></code> elements that share the same parent as the <code><h3></code> .

Descendant Selector:

Although some contextual selectors are not commonly used, descendant selectors are essential for almost all web authors. A descendant selector identifies all elements that are enclosed within another element. The space character is used to indicate descendant selection.

context selected element

`div p { ... }`



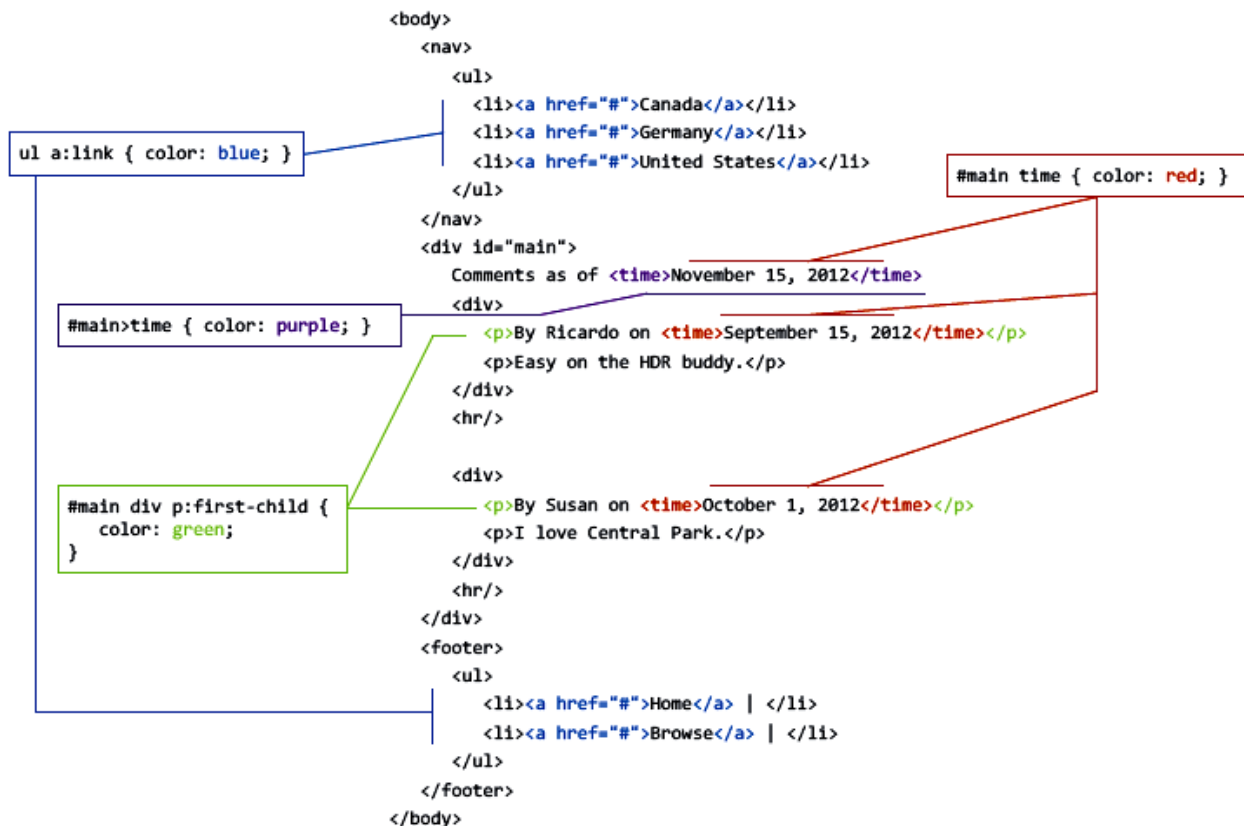
Selects a `<p>` element
somewhere
within a `<div>` element

`#main div p:first-child { ... }`



Selects the first `<p>` element
somewhere within a `<div>` element
that is somewhere within an element
with an `id="main"`

Contextual Selectors in Action



```
<body>
<nav>
<ul>
<li><a href="#">Canada</a></li>
<li><a href="#">Germany</a></li>
<li><a href="#">United States</a></li>
</ul>
</nav>
<div id="main">
Comments as of <time>November 15, 2012</time>
<div>
<p>By Ricardo on <time>September 15, 2012</time></p>
```

```
<p>Easy on the HDR buddy.</p>
</div>
<hr/>
<div>
<p>By Susan on <time>October 1, 2012</time></p>
<p>I love Central Park.</p>
</div>
<hr/>
</div>
<footer>
<ul>
<li><a href="#">Home</a> | </li>
<li><a href="#">Browse</a> | </li>
</ul>
</footer>
</body>
```

THE CASCADE: HOW STYLES INTERACT

Understanding the Reasons for Conflict in CSS

CSS, or Cascading Style Sheets, is a powerful tool for web developers to create visually appealing and consistent designs across their websites. However, conflicts can arise when different style rules clash with each other. It is important to understand the reasons behind these conflicts to effectively troubleshoot and resolve them.

One reason for conflict is the existence of three different types of style sheets: author-created, user-defined, and the default browser style sheet. Each of these style sheets can contain different rules for the same HTML element, leading to confusion for the browser on which rule to follow.

Another factor contributing to conflict is the ability of author-created style sheets to define multiple rules for the same HTML element. This can create ambiguity for the browser, resulting in unexpected or undesired display of the element.

Web-Technology and Applications (Module1)

Fortunately, CSS has a system in place to help the browser determine how to display elements when different style rules conflict. By using a cascading order of precedence, the browser can prioritize which rule to follow based on its specificity, importance, and origin.

In conclusion, understanding the reasons for conflict in CSS is crucial for web developers to create effective and visually appealing designs. By utilizing the cascading order of precedence, conflicts can be resolved and a consistent design can be achieved.

Cascade: How Conflicting Rules are handled in CSS

The term "Cascade" in CSS refers to the way conflicting rules are managed. The visual metaphor behind the term cascade is that of a mountain stream flowing downstream over rocks. The downward movement of water down a cascade is meant to be analogous to how a given style rule will continue to take precedence with child elements.

In simpler terms, the cascade determines which style rules take priority when there are multiple rules that apply to the same element. This is important because it allows developers to create complex and dynamic web pages with ease.

For example, if you have a paragraph element that has both a font-size rule and a color rule applied to it, the cascade will determine which rule takes precedence. If the font-size rule has a higher specificity or is defined later in the style sheet, it will take priority over the color rule.

Understanding the cascade is essential for creating well-organized and efficient style sheets. By following the cascade's rules, developers can ensure that their web pages look and function as intended, regardless of the complexity of the design.

Cascade Principles in CSS

Cascade Principles in CSS are designed to effectively manage conflicts that may arise. These principles include inheritance, specificity, and location.

Inheritance refers to the ability of child elements to inherit properties from their parent elements. This allows for a more efficient and streamlined approach to styling, as properties only need to be defined once at the parent level.

Specificity is another important principle, as it determines which styles take precedence when multiple styles are applied to the same element. The more specific a style is, the higher its priority.

Web-Technology and Applications (Module1)

Finally, **location** plays a role in determining which styles are applied. Styles defined later in the stylesheet will override styles defined earlier, unless the earlier styles have a higher specificity.

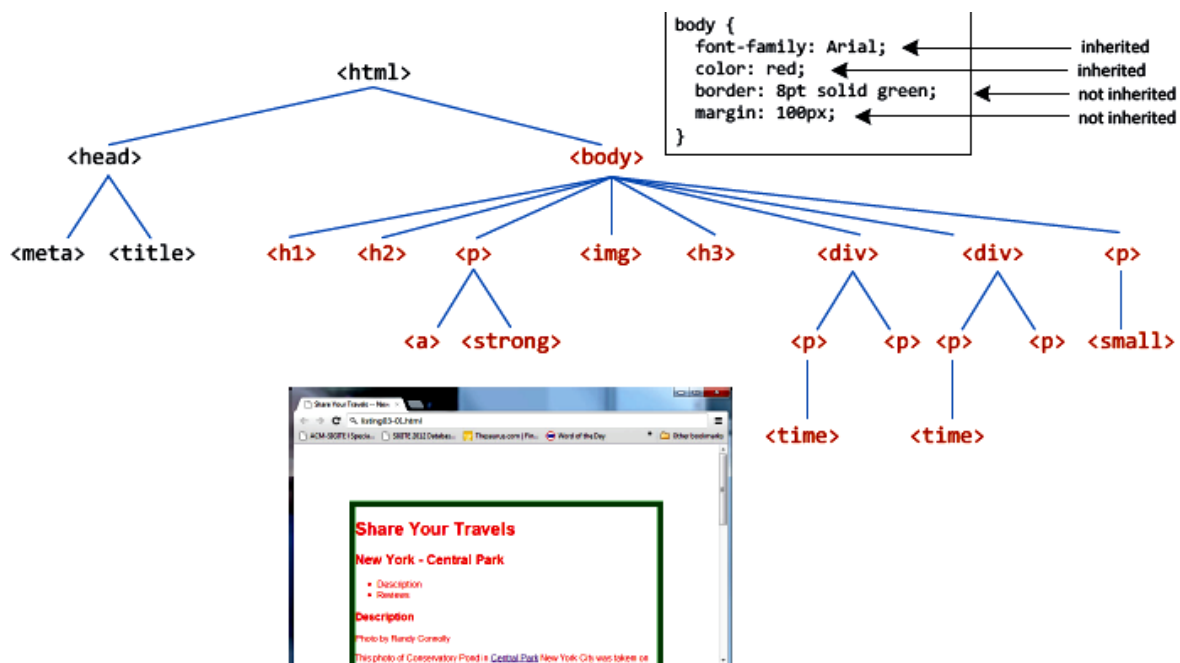
By utilizing these cascade principles, CSS is able to effectively manage conflicts and ensure that styles are applied consistently and efficiently throughout a website or application.

Inheritance

Inheritance is a fundamental concept in CSS that allows certain properties to be passed down from parent elements to their children. While not all CSS properties are inheritable, many of them affect not only the element they are applied to, but also their descendants.

For instance, font, color, list, and text properties are inheritable, meaning that any child elements will inherit these properties from their parent. On the other hand, layout, sizing, border, background, and spacing properties are not inheritable, and must be explicitly defined for each individual element.

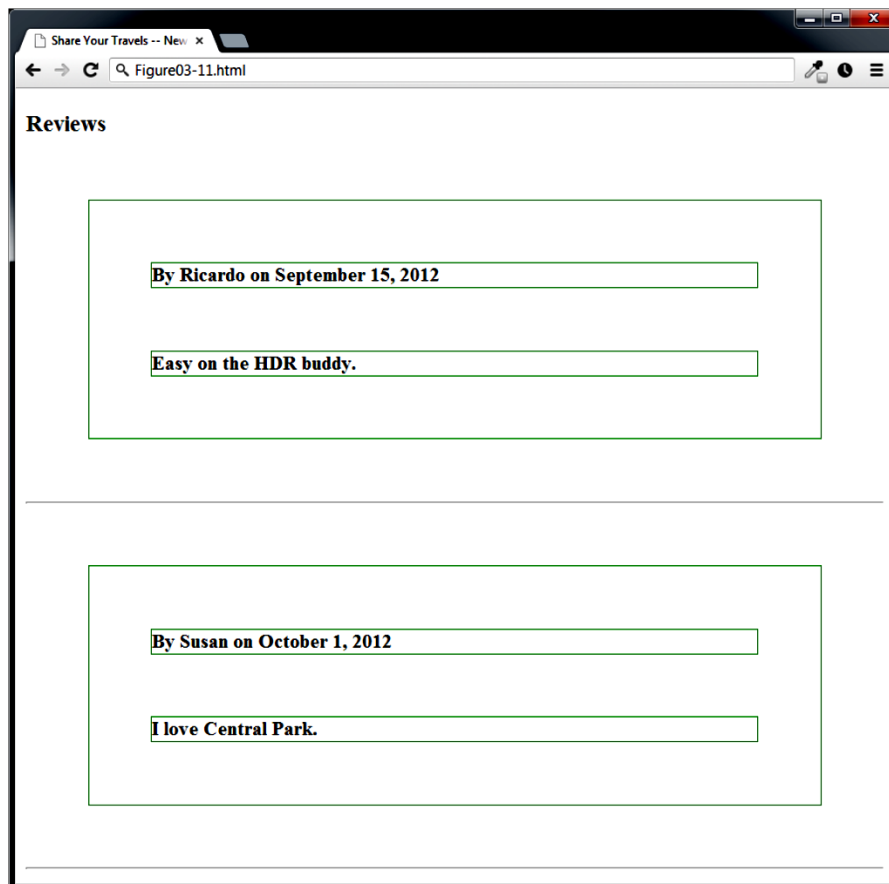
Understanding inheritance is crucial for creating efficient and maintainable CSS code. By utilizing inheritable properties, you can reduce the amount of code needed to style your elements, while also ensuring consistency throughout your design.

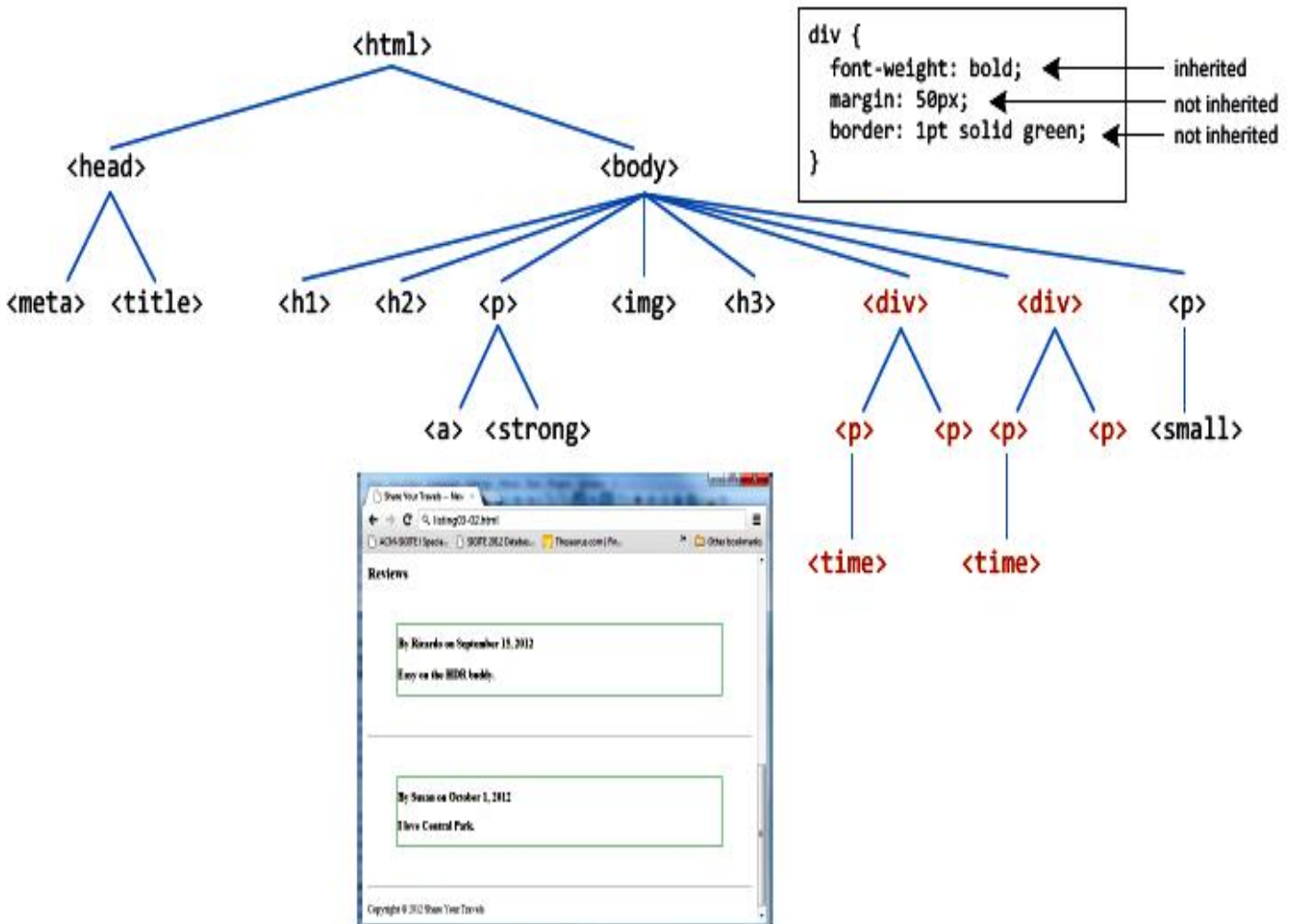


Web-Technology and Applications (Module1)

It is possible to tell elements to inherit properties that are normally not inheritable.

```
div {  
  font-weight: bold;  
  margin: 50px;  
  border: 1pt solid green;  
}  
p {  
  border: inherit;  
  margin: inherit;  
}  
<h3>Reviews</h3>  
<div>  
  <p>By Ricardo on <time>September 15, 2012</time></p>  
  <p>Easy on the HDR buddy.</p>  
</div>  
<hr/>  
<div>  
  <p>By Susan on <time>October 1, 2012</time></p>  
  <p>I love Central Park.</p>  
</div>  
<hr/>
```





Specificity

Specificity is a crucial concept in web design that determines which style rule takes precedence when multiple style rules could be applied to the same element. The more specific the selector, the more it overrides the previous definition.

To understand how specificity works in the browser, it's important to note that the browser assigns a weight to each style rule. When multiple rules apply, the one with the greatest weight takes precedence.

In other words, specificity is all about ensuring that the correct style rules are applied to the correct elements on a webpage. By using specific selectors and assigning appropriate weights to style rules, designers can ensure that their designs look polished and professional.

Web-Technology and Applications (Module1)

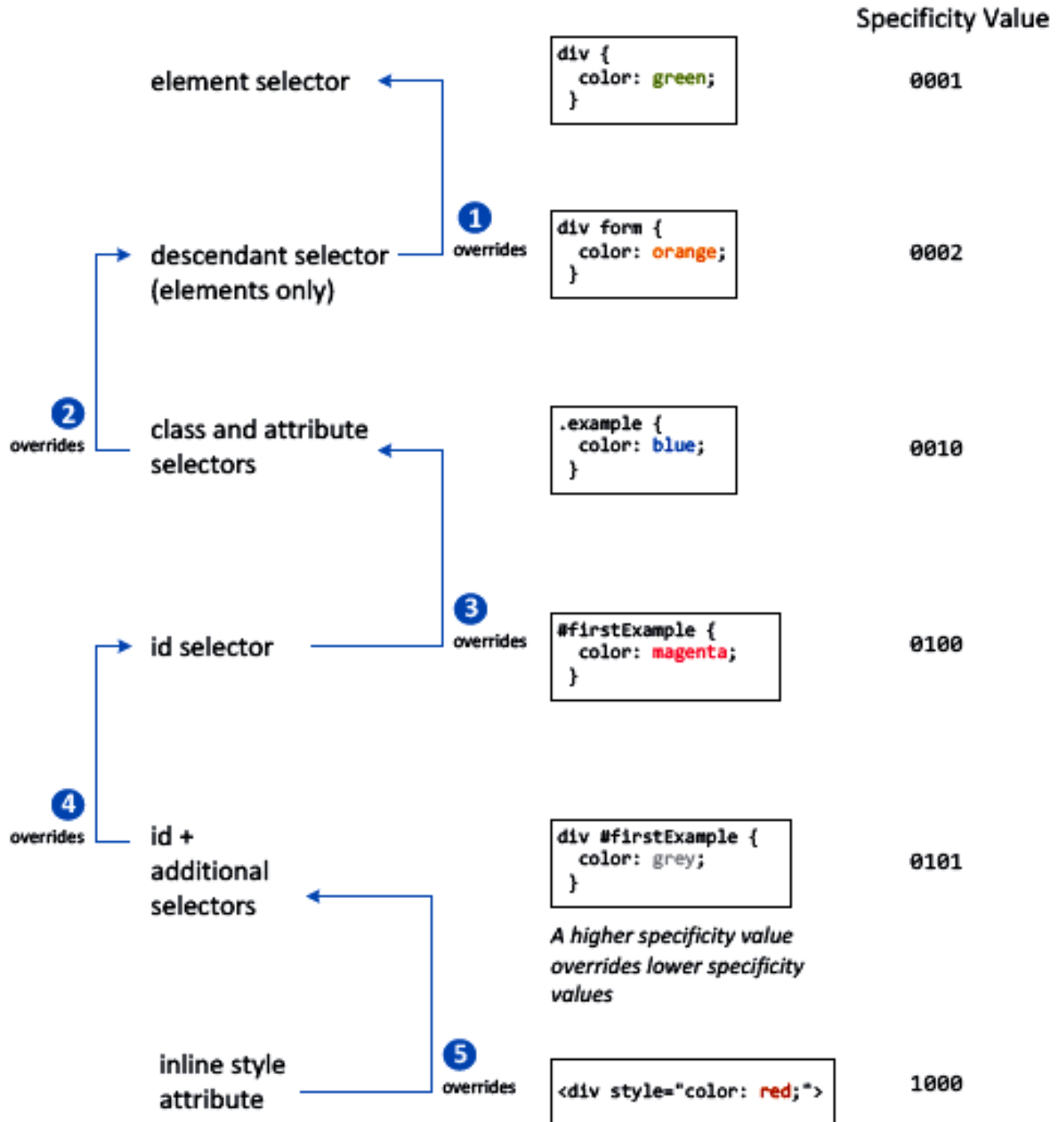
These color and font-weight properties are inheritable and thus potentially applicable to the entire child Elements contained within the body.	body { font-weight: bold; color: red; }	This text is not within a p element. <p>Reviews</p> <div> <p>By Ricardo on <time>September 15, 2012</time> <p>Easy on the HDR buddy.</p> This text is not within a p element. </div>
However, because the <div> and <p> elements also have the same properties set, they <i>override</i> the value defined for the <body> element because their selectors (div and p) are more specific	div { font-weight: normal; color: magenta; } p { color: green; }	 <div> <p>By Susan on <time>October 1, 2012</time></p> <p>I love Central Park.</p> </div> <div>
Class selectors are more specific than element selectors, and thus take precedence and override element selectors	.last { color: blue; }	<p class="last">By Dave on <time>October 15, 201</p> <p class="last" id="verylast">Thanks for posting</p> </div>
Id selectors are more specific than class selectors, and thus take precedence and override class selectors.	#verylast { color: orange; font-size: 16pt; }	

Specificity Algorithm

The Specificity Algorithm is a method used to calculate the specificity of a selector in CSS. To determine the specificity of a selector, follow these steps:

1. First, determine if the declaration is from a style attribute in the HTML. If it is, count 1. If not, count 0. Let this value be represented by "a".
2. Next, count the number of ID attributes in the selector. Let this value be represented by "b".
3. Count the number of other attributes and pseudo-classes in the selector. Let this value be represented by "c".
4. Count the number of element names and pseudo-elements in the selector. Let this value be represented by "d".
5. Finally, concatenate the four numbers (a + b + c + d) together to calculate the selector's specificity.

By using the Specificity Algorithm, you can accurately determine the specificity of a selector and ensure that your CSS styles are applied correctly.



Location:

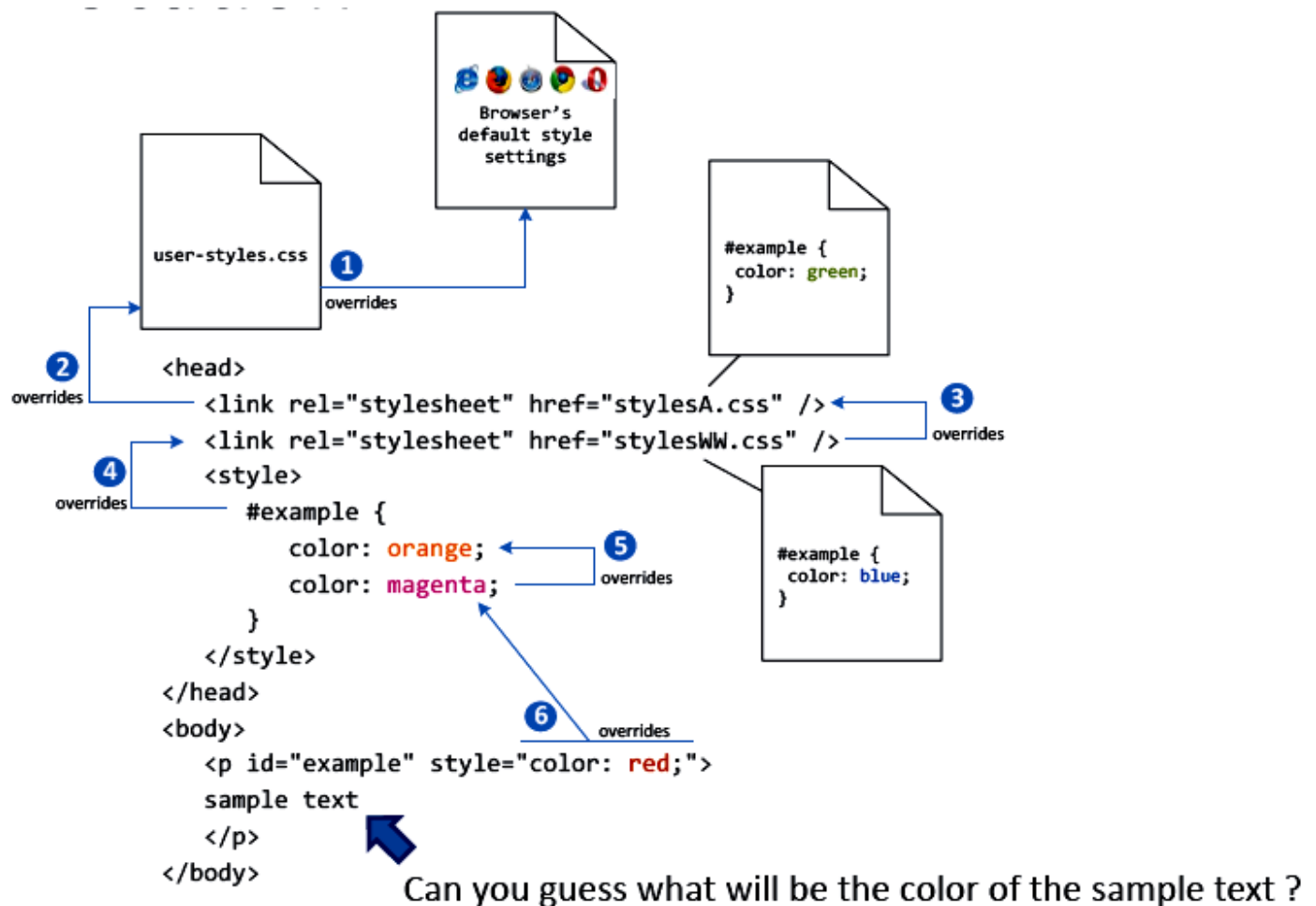
When inheritance and specificity fail to determine style precedence, the principle of location comes into play. The principle of location states that when rules have the same level of

Web-Technology and Applications (Module1)

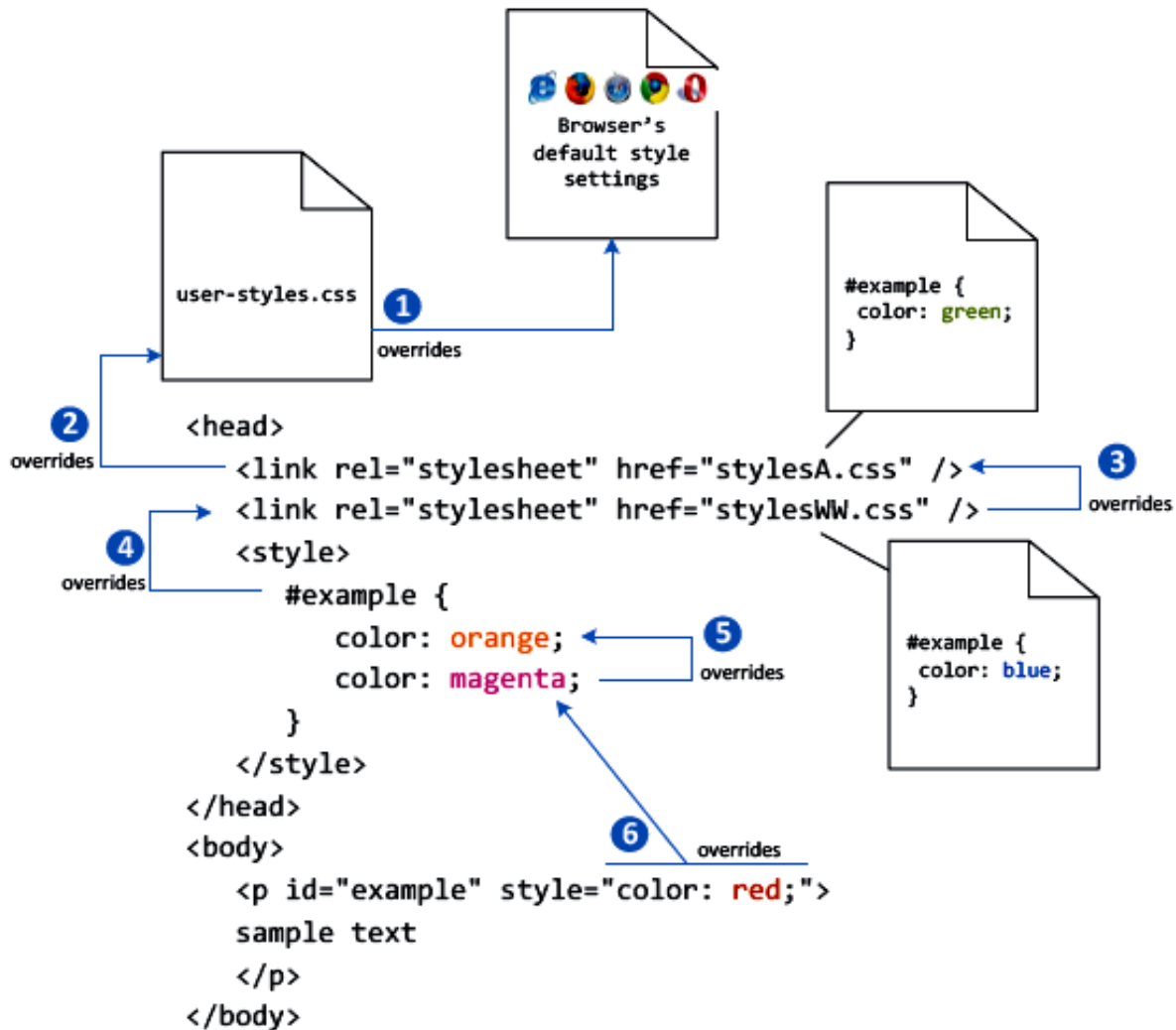
specificity, the latest rules are given more weight. This means that the most recently declared rule will take precedence over any previously declared rules.

For example, if you have two CSS rules that apply to the same element and have the same level of specificity, the rule that appears later in the stylesheet will be applied. This principle is particularly useful when dealing with complex stylesheets that have multiple rules that apply to the same element.

By using the principle of location, you can ensure that your styles are applied consistently and in the correct order. It is important to keep in mind that the principle of location should only be used as a last resort when other methods of determining style precedence have failed.



What color would the sample text be if there wasn't an inline style definition?



There is a notable exception to the principle of location when it comes to author-created style rules. If a property is marked with the **"! important"** tag, it will take precedence over any other author-created style, regardless of its location. However, there is one exception to this exception: if a user style sheet contains a rule marked with **"! important,"** it will override all other rules. It's important to keep this in mind when designing and implementing style rules for your website or application.

The Box Model

The Box Model is a fundamental concept in CSS, where all HTML elements are contained within an element box. It is crucial to understand the terminology and relationship of the CSS

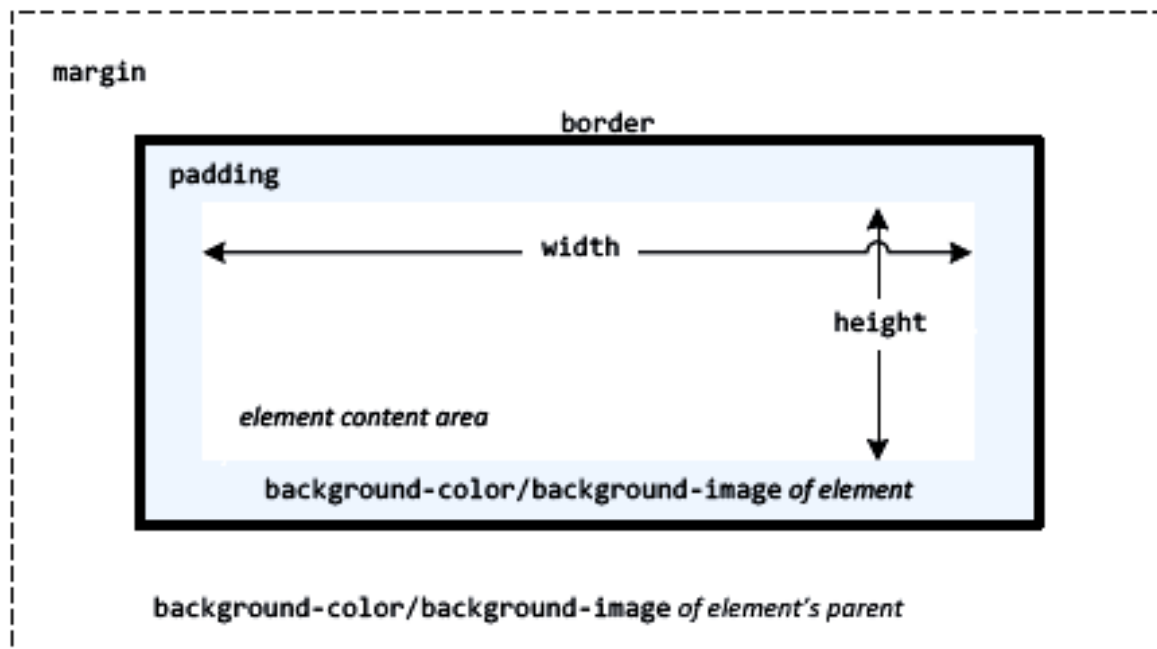
Web-Technology and Applications (Module1)

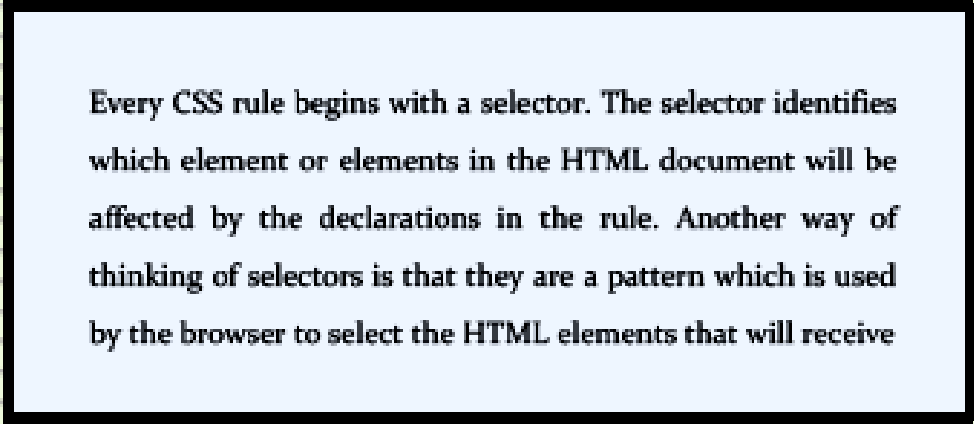
properties within this box. By doing so, you can effectively control the layout and design of your web page.

Think of the element box as a container that surrounds your HTML content. It consists of four main components: the content area, padding, border, and margin. The content area is where your text and images are displayed, while the padding is the space between the content and the border. The border is the line that surrounds the content and padding, and the margin is the space between the border and the surrounding elements.

To manipulate the element box, you can use CSS properties such as width, height, padding, border, and margin. By adjusting these properties, you can control the size, spacing, and positioning of your HTML elements.

In conclusion, understanding the Box Model is essential for creating visually appealing and functional web pages. By thinking inside the box, you can effectively control the layout and design of your content.





Every CSS rule begins with a selector. The selector identifies which element or elements in the HTML document will be affected by the declarations in the rule. Another way of thinking of selectors is that they are a pattern which is used by the browser to select the HTML elements that will receive

Background:

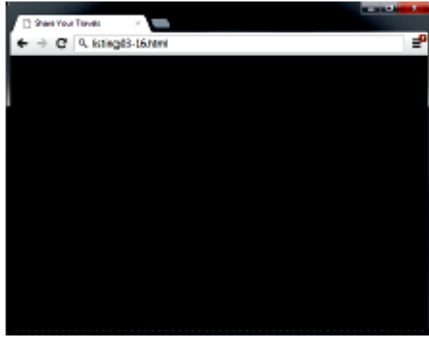
The color or image that fills an element up to its border is known as the background. In modern web design, it is prevalent to use CSS for displaying purely presentational images, such as background gradients, patterns, and decorative images, instead of using the element.

The background of an element is an essential aspect of web design. It can enhance the overall look and feel of a website, making it more visually appealing and engaging for users. By using CSS, designers can create unique and eye-catching backgrounds that add depth and dimension to a website.

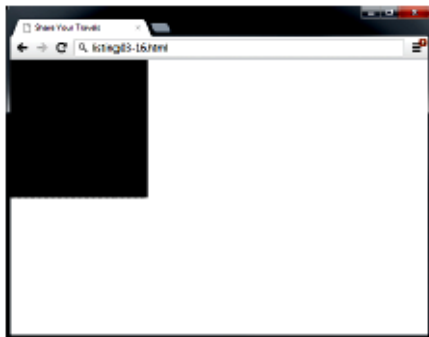
Moreover, using CSS for background images is more efficient than using the element. It reduces the number of HTTP requests, which can improve the website's loading speed and performance. Additionally, CSS allows for more flexibility and control over the design, enabling designers to create backgrounds that are responsive and adaptable to different screen sizes.

In conclusion, the background is a crucial element of web design that can significantly impact a website's aesthetics and functionality. By utilizing CSS for background images, designers can create visually stunning and efficient websites that provide an optimal user experience.

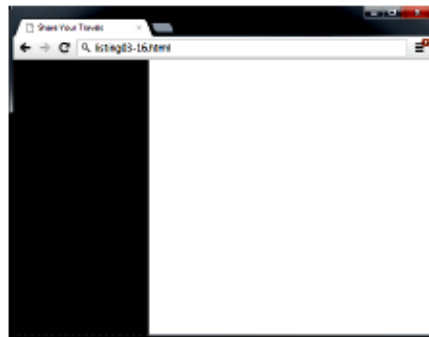
Property	Description
background	A combined short-hand property that allows you to set the background values in one property. While you can omit properties with the short-hand, do remember that any omitted properties will be set to their default value.
background-attachment	Specifies whether the background image scrolls with the document (default) or remains fixed. Possible values are: fixed, scroll.
background-color	Sets the background color of the element.
background-image	Specifies the background image (which is generally a jpeg, gif, or png file) for the element. Note that the URL is relative to the CSS file and not the HTML. CSS3 introduced the ability to specify multiple background images.
background-position	Specifies where on the element the background image will be placed. Some possible values include: bottom, center, left, and right. You can also supply a pixel or percentage numeric position value as well. When supplying a numeric value, you must supply a horizontal/vertical pair; this value indicates its distance from the top left corner of the element.
background-repeat	Determines whether the background image will be repeated. This is a common technique for creating a tiled background (it is in fact the default behavior). Possible values are: repeat, repeat-x, repeat-y, and no-repeat.
background-size	New to CSS3, this property lets you modify the size of the background image.



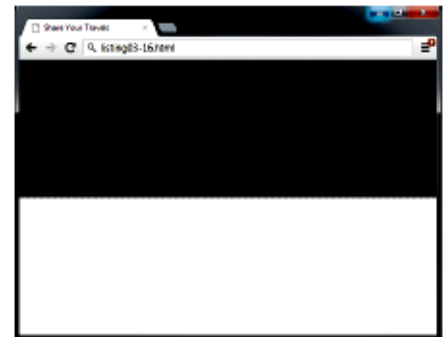
`background-image: url(../images/backgrounds/body-background-tile.gif);`
`background-repeat: repeat;`



`background-repeat: no-repeat;`

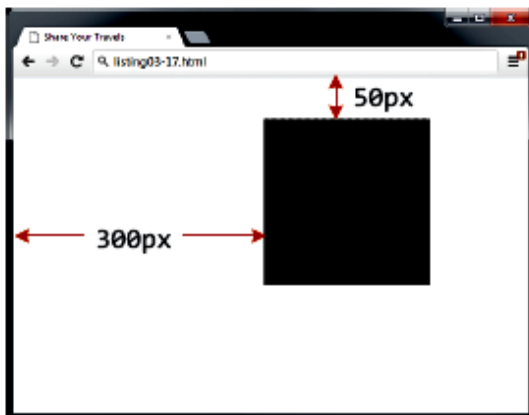


`background-repeat: repeat-y;`



`background-repeat: repeat-x;`

Background Position



```
body {  
    background: white url(../images/backgrounds/body-background-tile.gif) no-repeat;  
    background-position: 300px 50px;  
}
```

Borders

Borders serve as a visual tool to distinguish between different elements on a page. They can be applied to all four sides of an element or selectively to one, two, or three sides. Borders are a simple yet effective way to enhance the overall design and organization of a webpage. By utilizing borders, you can create a more polished and professional appearance for your website.

Property	Description
border	A combined short-hand property that allows you to set the style, width, and color of a border in one property. The order is important and must be: border-style border-width border-color
border-style	Specifies the line type of the border. Possible values are: solid, dotted, dashed, double, groove, ridge, inset, and outset.
border-width	The width of the border in a unit (but not percents). A variety of keywords (thin, medium, etc) are also supported.
border-color	The color of the border in a color unit.
border-radius	The radius of a rounded corner.
border-image	The URL of an image to use as a border.

Shortcut notation

With border, margin, and padding properties, there are long form and shortcut methods to set the

4 sides

TRBL

`border-top-color: red; /* sets just the top side */`

`border-right-color: green; /* sets just the right side */`

`border-bottom-color: yellow; /* sets just the bottom side */`

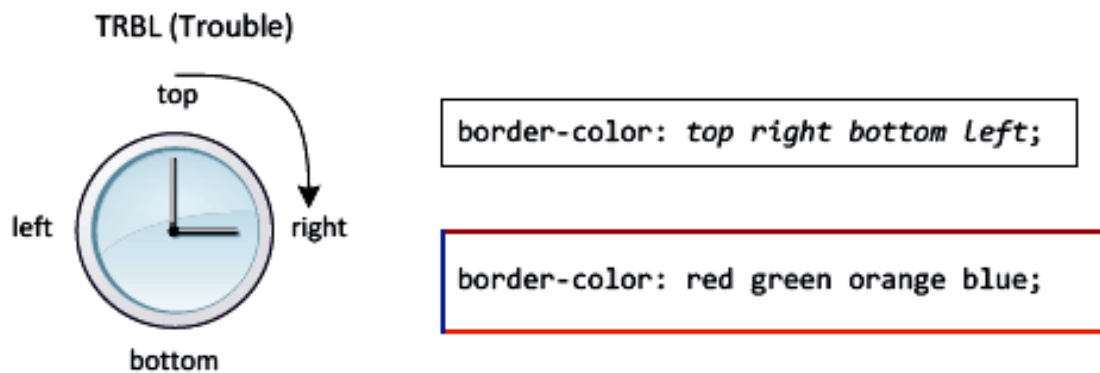
`border-left-color: blue; /* sets just the left side */`

`border-color: red; /* sets all four sides to red */`

`border-color: red green orange blue; /* sets all four sides differently */`

When using this multiple values shortcut, they are applied in clockwise order starting at the top.

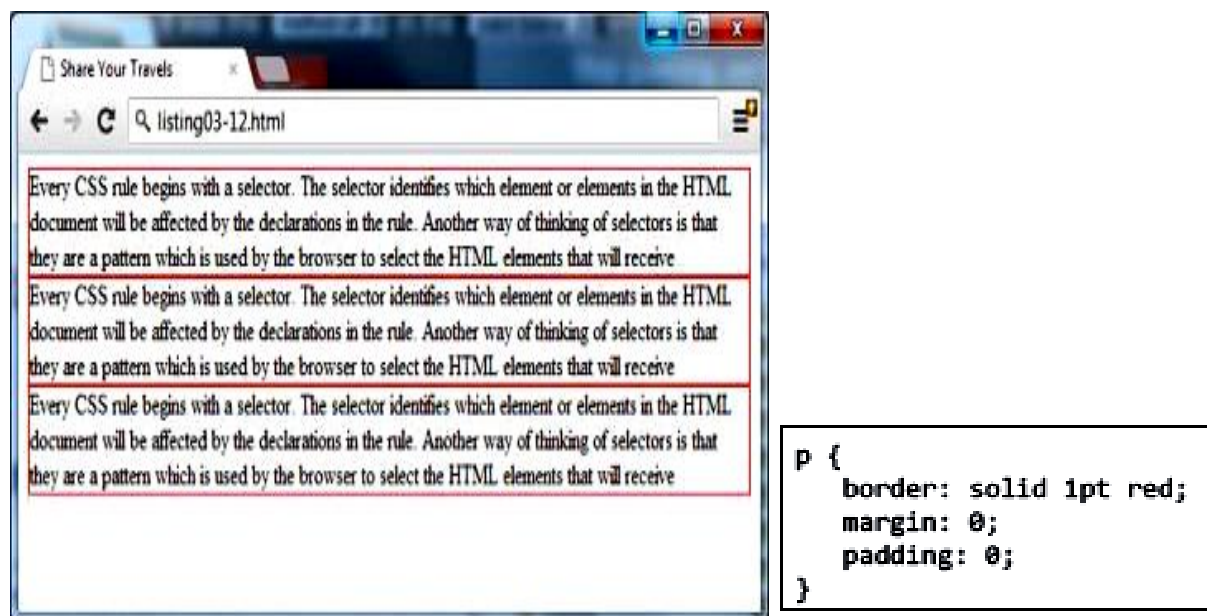
Thus the order is: **top right bottom left**.

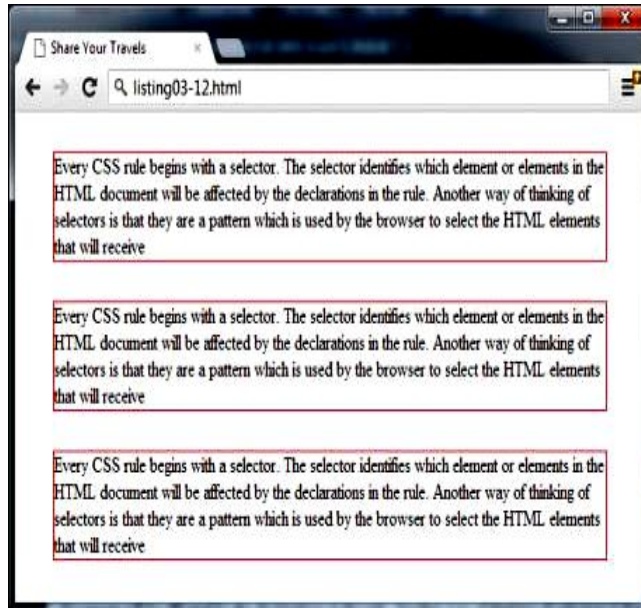


Margins

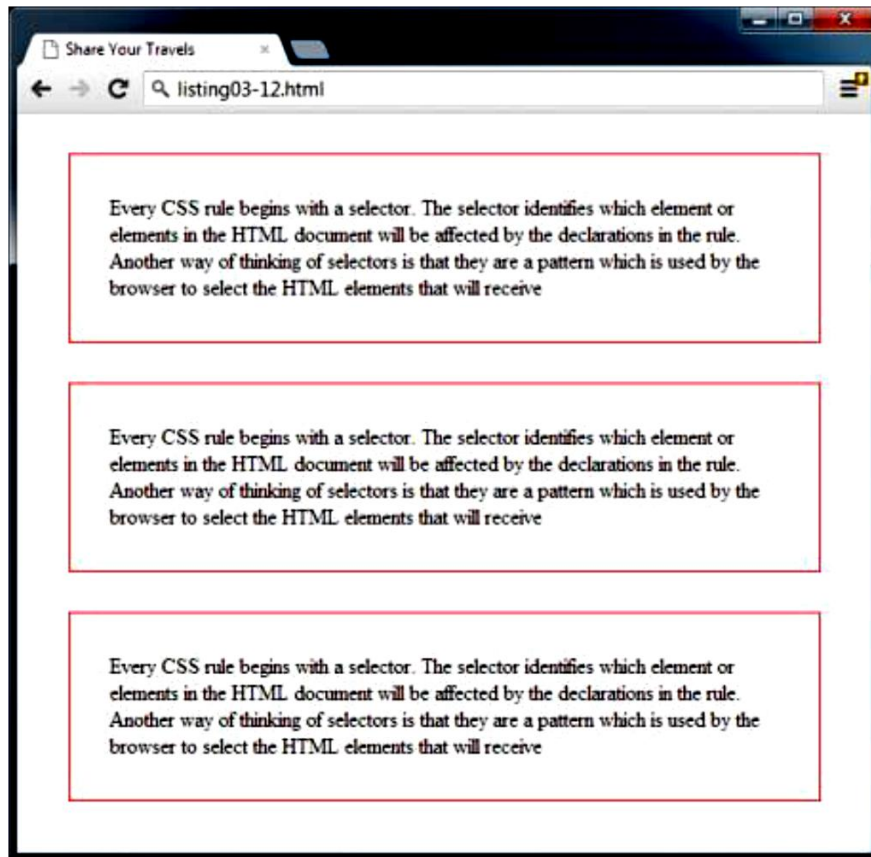
Margins play an important role in the formatting of written content. Have you ever noticed that the space between paragraphs one and two, and two and three, is the same as the space before paragraph one and after paragraph three? This is because adjoining vertical margins have a tendency to collapse.

In simpler terms, when two margins are placed next to each other, they tend to merge into one. This can be a bit confusing for those who are new to formatting, but it is an important concept to understand in order to create visually appealing and professional-looking documents. By paying attention to margins and their behavior, you can ensure that your written content is easy to read and aesthetically pleasing.





```
p {  
  border: solid 1pt red;  
  margin: 30px;  
  padding: 0;  
}
```



```
p {  
  border: solid 1pt red;  
  margin: 30px;  
  padding: 30px;  
}
```

Collapsing margins

Collapsing margins can be a tricky concept to grasp, but it's an important one to understand in web design. When two elements have vertical margins that touch, the larger margin value will be displayed while the smaller margin value will be collapsed to zero. It's important to note that horizontal margins never collapse.

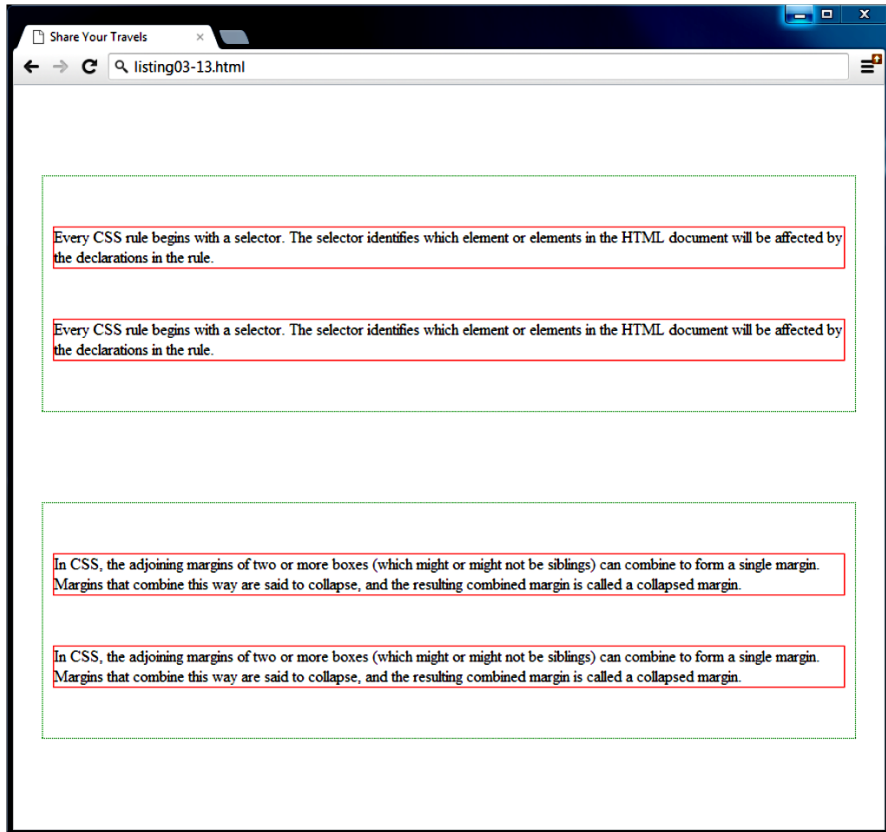
However, it's not always that simple. There are numerous special cases in which adjoining vertical margins do not collapse. This can be frustrating for designers, but it's important to be aware of these exceptions to ensure that your website looks and functions as intended.

In summary, collapsing margins occur when two elements have vertical margins that touch, and the larger margin value is displayed while the smaller margin value is collapsed to zero. While horizontal margins never collapse, there are exceptions to the rule when it comes to vertical margins. By understanding these nuances, you can create a polished and professional website that looks great and functions seamlessly.

```
<div>
  <p>Every CSS rule ...</p>
  <p>Every CSS rule ...</p>
</div>
<div>
  <p>In CSS, the adjoining ... </p>
  <p>In CSS, the adjoining ... </p>
</div>
```

```
div {
  border: dotted 1pt green;
  padding: 0;
  margin: 90px 20px;
}
```

```
p {
  border: solid 1pt red;
  padding: 0;
  margin: 50px 20px;
}
```



Width and Height:

The width and height properties are used to specify the size of an element's content area. While these properties may seem straightforward, they can present a number of potential issues that can be troubling for students.

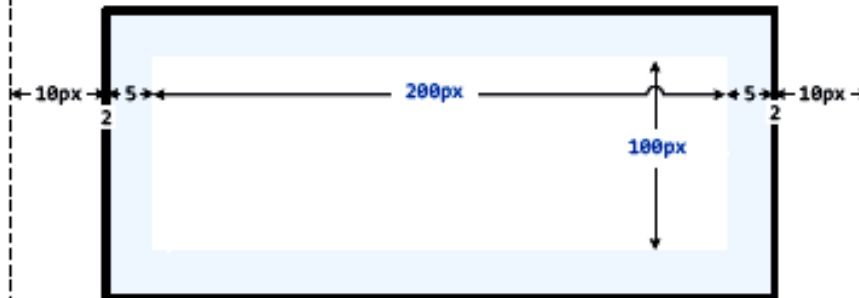
It's important to note that only block-level elements and non-text inline elements, such as images, have a width and height that can be specified. By default, the width and height of an element is determined by the actual size of its content. For text, this is determined by the font size and font face, while for images, the width and height of the actual image in pixels determines the element box's dimensions.

It's worth noting that the total size of an element is not just its content area, but also the sum of its padding, borders, and margins. This means that the width and height properties refer specifically to the size of the content area, and not the total size of the element.

While the width and height properties may seem simple, they can present a number of potential issues that can be difficult to troubleshoot. By understanding these properties and their potential issues, students can create more effective and professional-looking web designs.

```
div {  
  box-sizing: content-box;  
  width: 200px;  
  height: 100px;  
  padding: 5px;  
  margin: 10px;  
  border: solid 2pt black;  
}
```

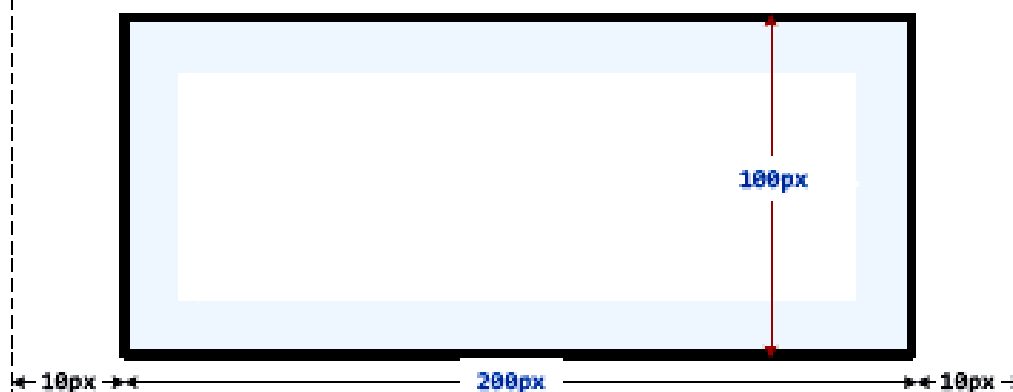
True element width = $10 + 2 + 5 + 200 + 5 + 2 + 10 = 234$ px
True element height = $10 + 2 + 5 + 100 + 5 + 2 + 10 = 134$ px



Default

```
div {  
  ...  
  box-sizing: border-box;  
}
```

True element width = $10 + 200 + 10 = 220$ px
True element height = $10 + 100 + 10 = 120$ px

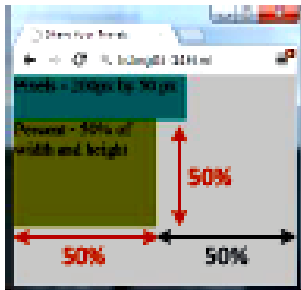


Sizing Elements:

In modern web design, many designers opt to use percentages or em units for measuring the width and height of elements, as opposed to pixels. When using percentages, the size of the element is relative to the size of its parent element. On the other hand, when using ems, the size of the element is relative to the size of the text within it.

The primary reason for using these relative measures is to ensure that the design remains scalable across different browsers and devices. By using percentages or em units, designers can create a flexible layout that adapts to the user's screen size, resulting in a more user-friendly experience.

In summary, when designing a website, it is essential to consider the sizing of elements carefully. By using relative measures such as percentages or em units, designers can create a scalable and adaptable design that works seamlessly across different devices and browsers.



```
<body>
  <div class="pixels">
    Pixels - 200px by 50 px
  </div>
  <div class="percent">
    Percent - 50% of width and height
  </div>
</body>
```

Developer Tools

Developer tools available in modern browsers have made it much simpler to examine and troubleshoot CSS compared to a decade ago. By utilizing the CSS inspection tools provided by different browsers, you can easily examine the box values of a selected element, among other things.

These tools have revolutionized the way developers work, allowing them to quickly identify and fix issues with their CSS code. With the ability to inspect and modify CSS in real-time, developers can now create more efficient and visually appealing websites.

In conclusion, the availability of developer tools in modern browsers has greatly improved the CSS development process, making it easier and more efficient for developers to create high-quality websites.

TEXT STYLING

Text Properties

CSS offers two types of properties that impact text. The first type is font properties, which influence the font and its visual presentation. The second type is paragraph properties, which affect the text in a consistent manner, regardless of the font being utilized.

Font properties include attributes such as font-family, font-size, font-style, and font-weight. These properties allow you to customize the font type, size, style, and weight to suit your design needs.

On the other hand, paragraph properties include line-height, text-align, text-indent, and text-decoration. These properties enable you to adjust the spacing, alignment, indentation, and decoration of your text.

By utilizing these text properties, you can enhance the readability and visual appeal of your website's content. Whether you want to create a sleek and modern look or a classic and elegant feel, CSS text properties provide the flexibility and customization options you need to achieve your desired outcome.

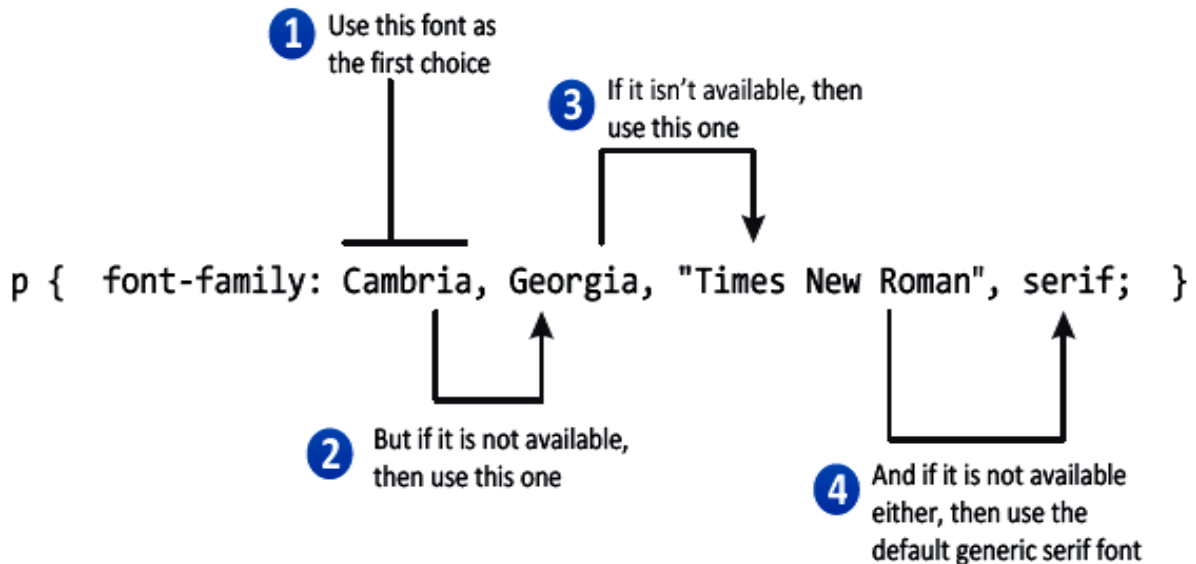
Font Family:

When using a word processor on a desktop machine, you have the freedom to choose any font that is installed on your computer. However, when it comes to web development, it's not that simple. Just because a font is available on the developer's computer, it doesn't mean that it will be available for all users who view the site.

To ensure that your website looks consistent across all devices and browsers, it's important to provide a web font stack. This is a series of alternate fonts that can be used in case the original font choice is not available on the user's computer.

By providing a web font stack, you can ensure that your website looks professional and polished, no matter what device or browser your users are using. So, take the time to choose the right font family for your website and provide a web font stack to ensure that your website looks its best.

Specifying the Font-Family



	Generic Font-Family Name	
This	serif	serif Th
This	sans-serif	Without ("sans") serif Th
This	monospace	In a monospace font, each letter has the same width This
This	cursive	
This	fantasy	Decorative and cursive fonts vary from system to system; rarely used as a result.

In recent years, the latest versions of web browsers have started to support the @font-face selector in CSS. This selector enables you to use a font on your website, even if it is not installed

on the user's computer. This is a significant development, as it allows designers to use a wider range of fonts, without worrying about compatibility issues.

Thanks to the growing popularity of open-source font sites like Google Web Fonts and Font Squirrel, @font-face has become widely used. This is because these sites offer a vast selection of fonts that can be easily integrated into your website. As a result, @font-face has reached a critical mass of usage, making it an essential tool for web designers.

Font Sizes

The topic of font sizes can be quite tricky, especially when it comes to designing for different devices. While specifying a font size in points may work well for print-based programs like word processors, it doesn't always translate well to pixel-based devices. Even pixels can be problematic, as newer mobile devices have increased pixel densities, making it difficult to correlate a CSS pixel to a single device pixel.

To create web layouts that work well on different devices, it's important to use relative units like em units or percentages for font sizes (and other sizes in CSS). This not only ensures consistency across devices, but also allows users to easily adjust the size of the text to their liking.

As one of the principles of the web, users should have the ability to change the size of the text if they choose to do so. By using percentages or em units, we can ensure that this user action will work seamlessly. So, when designing for the web, remember to keep font sizes in mind and use relative units for the best results.

<code><body></code>	Browser's default text size is usually 16 pixels
<code><p></code>	100% or 1em is 16 pixels
<code><h3></code>	125% or 1.125em is 18 pixels
<code><h2></code>	150% or 1.5em is 24 pixels
<code><h1></code>	200% or 2em is 32 pixels

```
/* using 16px scale */  
body { font-size: 100%; }  
h3 { font-size: 1.125em; } /* 1.25 x 16 = 18 */  
h2 { font-size: 1.5em; } /* 1.5 x 16 = 24 */  
h1 { font-size: 2em; } /* 2 x 16 = 32 */
```

```
<body>  
<p>this will be about 16 pixels</p>  
<h1>this will be about 32 pixels</h1>  
<h2>this will be about 24 pixels</h2>  
<h3>this will be about 18 pixels</h3>  
<p>this will be about 16 pixels</p>  
</body>
```

Using EMS and Percentages: A Guide

Although using EMS and percentages may seem simple at first glance, it can quickly become complicated. It's important to keep in mind that these units are relative to their parent elements. This means that if the parent font size changes, it will affect all of its contents.

To avoid confusion, it's best to use these units consistently throughout your code. This will ensure that your website or application is responsive and adaptable to different screen sizes and devices.

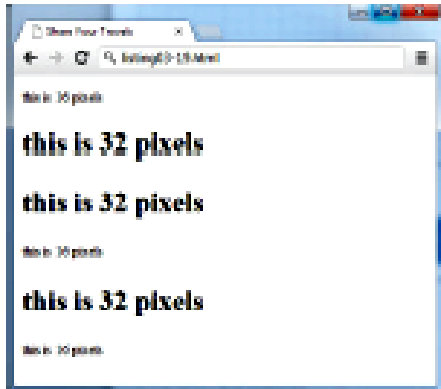
When using percentages, it's important to remember that they are based on the width of the parent element. This means that if the parent element changes size, the percentage will adjust accordingly.

On the other hand, EMS are based on the font size of the parent element. This can be useful for creating consistent spacing and sizing throughout your design.

Overall, using EMS and percentages can be a powerful tool for creating responsive and adaptable designs. By understanding how these units work, you can create a more professional and polished website or application.

Example:

```
<body>
<p>this is 16 pixels</p>
<h1>this is 32 pixels</h1>
<article>
<h1>this is 32 pixels</h1>
<p>this is 16 pixels</p>
<div>
<h1>this is 32 pixels</h1>
<p>this is 16 pixels</p>
</div>
</article>
</body>
```



/ using 16px scale */*

```
body { font-size: 100%; }  
p    { font-size: 1em; }    /* 1 x 16 = 16px */  
h1   { font-size: 2em; }    /* 2 x 16 = 32px */
```

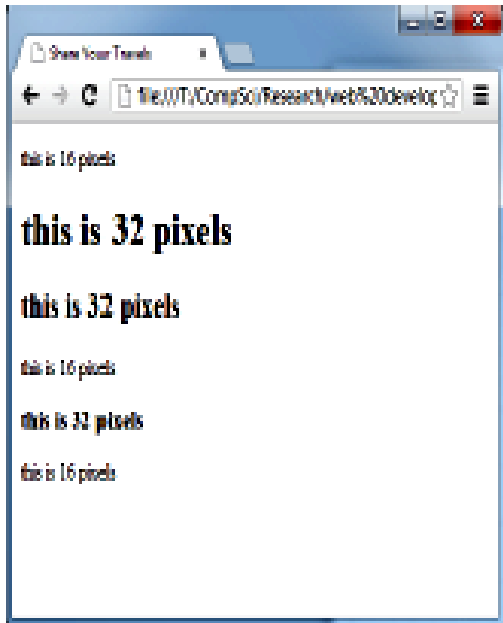


/ using 16px scale */*

```
body { font-size: 100%; }  
p    { font-size: 1em; }  
h1   { font-size: 2em; }  
  
article { font-size: 75% } /* h1 = 2 * 16 * 0.75 = 24px  
                           p = 1 * 16 * 0.75 = 12px */  
  
div   { font-size: 75% }  /* h1 = 2 * 16 * 0.75 * 0.75 = 18px  
                           p = 1 * 16 * 0.75 * 0.75 = 9px  */
```

The rem unit

CSS3 introduces a new relative measure known as the "rem" or "root em unit". This unit is always relative to the size of the root element, which is typically the <html> element. It's important to note that older versions of Internet Explorer (prior to version 9) do not support the rem unit, so it's necessary to provide a fallback for those browsers.



/ using 16px scale */*

```
body { font-size: 100%; }
p {
    font-size: 16px; /* for older browsers: won't scale properly though */
    font-size: 1rem; /* for new browsers: scales and simple too */
}
h1 { font-size: 2em; }

article { font-size: 75% } /* h1 = 2 * 16 * 0.75 = 24px
                           p = 1 * 16 = 16px */

div { font-size: 75% }     /* h1 = 2 * 16 * 0.75 * 0.75 = 18px
                           p = 1 * 16 = 16px */
```